

INFS3604

Business Process Management Week: 3 Advanced Process Modelling

The slides in this presentation have been adapted from supporting materials provided with the text: Dumas M. et al., 2018, Fundamentals of Business Process Management, Springer Verlag Berlin Heidelberg

Lecturer: George Joukhadar
E-mail: g.joukhadar@unsw.edu.au

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of New South Wales in accordance with section 113P(1) of the Copyright Act 1968 (Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

Copyright

- There are some file-sharing websites that specialise in buying and selling academic work to and from university students.
- If you upload your original work to these websites, and if another student downloads and presents it as their own either wholly or partially, **you might be found guilty of collusion — even years after graduation.**
- These file-sharing websites may also accept purchase of course materials, **such as copies of lecture slides and tutorial handouts. By law, the copyright on course materials, developed by UNSW staff in the course of their employment, belongs to UNSW. It constitutes copyright infringement, if not academic misconduct, to trade these materials.**

Country

Acknowledgement of Country

UNSW Business School acknowledges the Bidjigal (Kensington campus) and Gadigal (City campus) the traditional custodians of the lands where each campus is located.

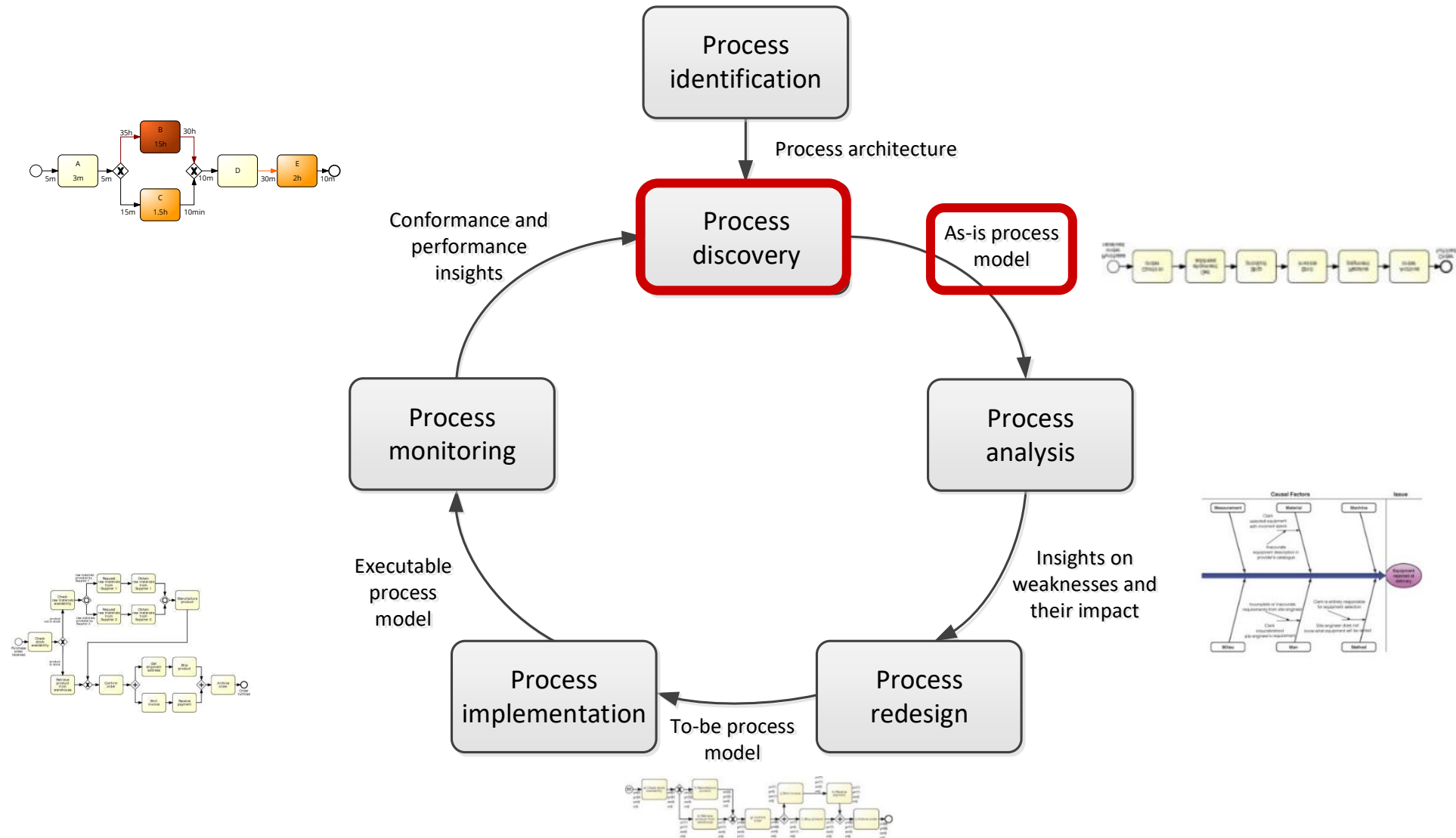
We acknowledge all Aboriginal and Torres Strait Islander Elders, past and present and their communities who have shared and practiced their teachings over thousands of years including business practices.

We recognise Aboriginal and Torres Strait Islander people's ongoing leadership and contributions, including to business, education and industry.



UNSW Business School. (2021, July 24). *Acknowledgement of Country* [online video]. Retrieved from <https://vimeo.com/369229957/d995d8087f>

Process Modeling in the BPM Lifecycle



Learning Outcomes

- Understand and use sub-processes to produce quality process models
- Appreciate the value of documenting processes using hierarchical (sub-process) structures.
- Understand and implement BPMN 2.0 notation for sub-processes.

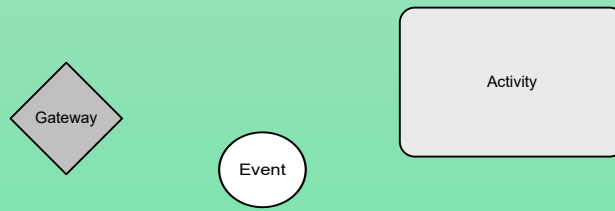
Week 3: Advanced Process Modeling

Contents

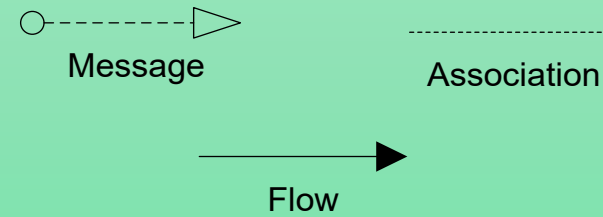
- BPMN main elements (Recap of week 2)
- Process Decomposition (Chp 3)
- Rework and Repetition (Chp 4)
- Handling Events (Chp 4)
- Handling Exceptions (Chp 4)
- Recap

BPMN Main Elements - Recap

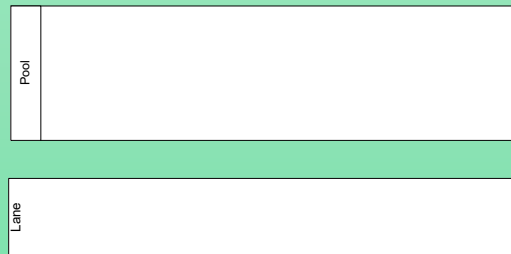
Flow Objects



Connections



Pools & lanes



Artifacts



BPMN Gateways

Exclusive (XOR)	Parallel (AND)	Inclusive (OR)
<u>Exclusive decision</u> take one branch	<u>Parallel split</u> take all branches	<u>Inclusive decision</u> take one or several branches depending on conditions
<u>Exclusive merge</u> Proceed when one branch has completed	<u>Parallel join</u> proceed when all incoming branches have completed	<u>Inclusive merge</u> proceed when all <u>active</u> incoming branches have completed

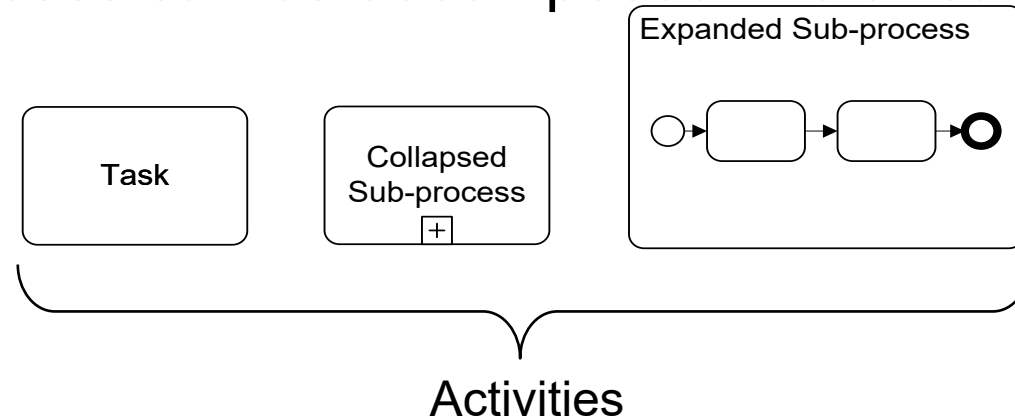
Week 3: Advanced Process Modeling

Contents

- BPMN main elements (Recap of week 2)
- **Process Decomposition (Chp 3)**
- Rework and Repetition (Chp 4)
- Handling Events (Chp 4)
- Handling Exceptions (Chp 4)
- Recap

Process decomposition

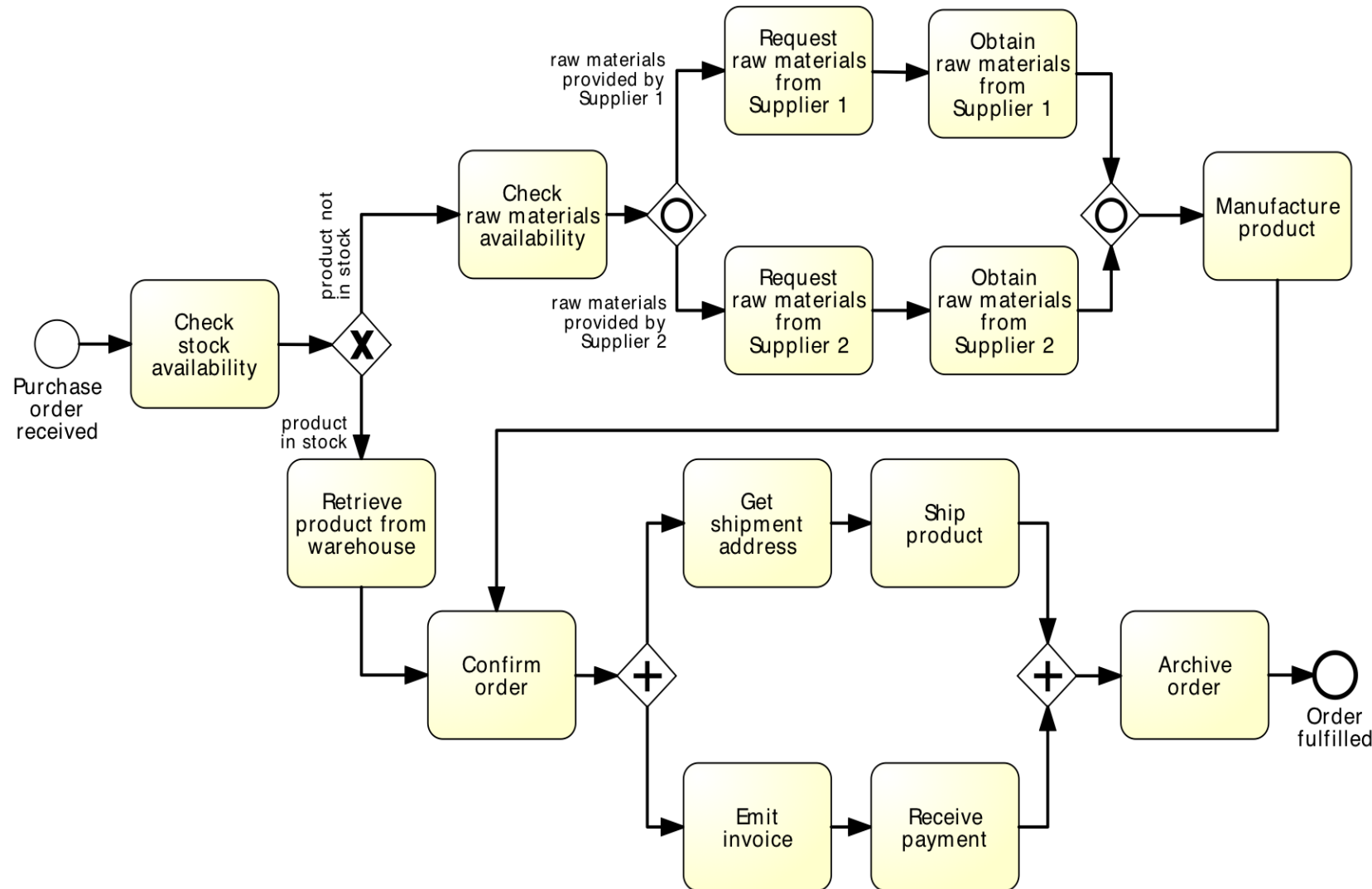
An activity in a process can be decomposed into a “sub-process”



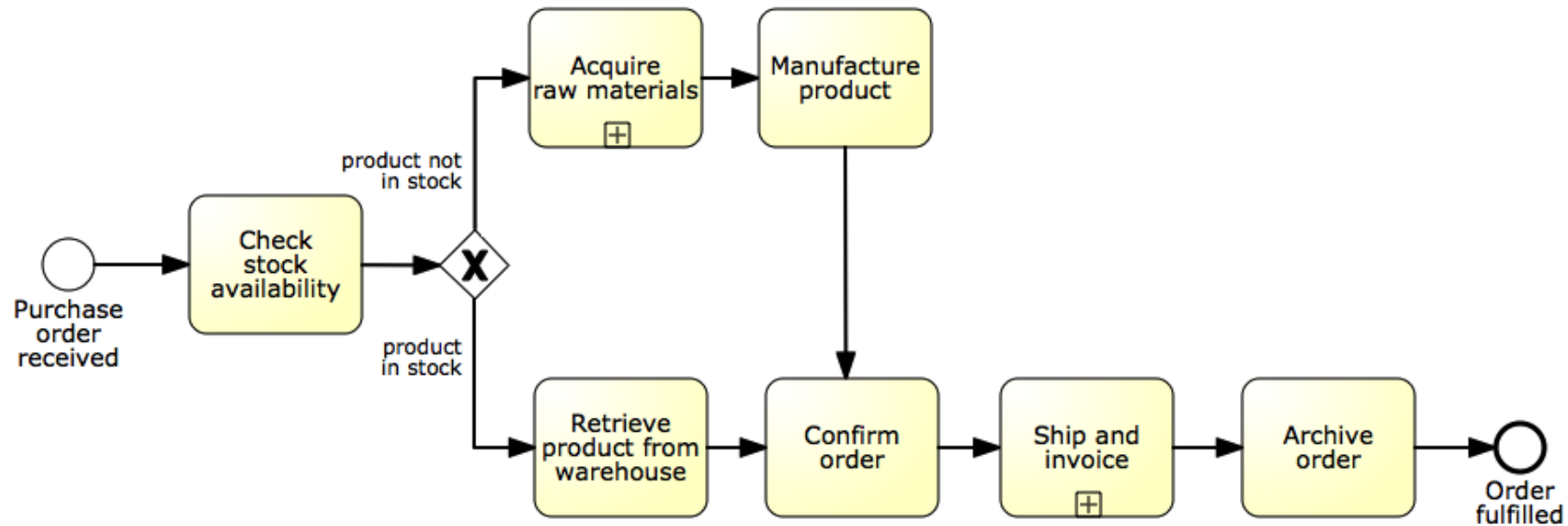
Use this feature to:

1. Improve understanding by breaking down large models
2. Identify parts that should be:
 - repeated
 - executed multiple times in parallel
 - interrupted, or
 - compensated

Anything wrong with this model?

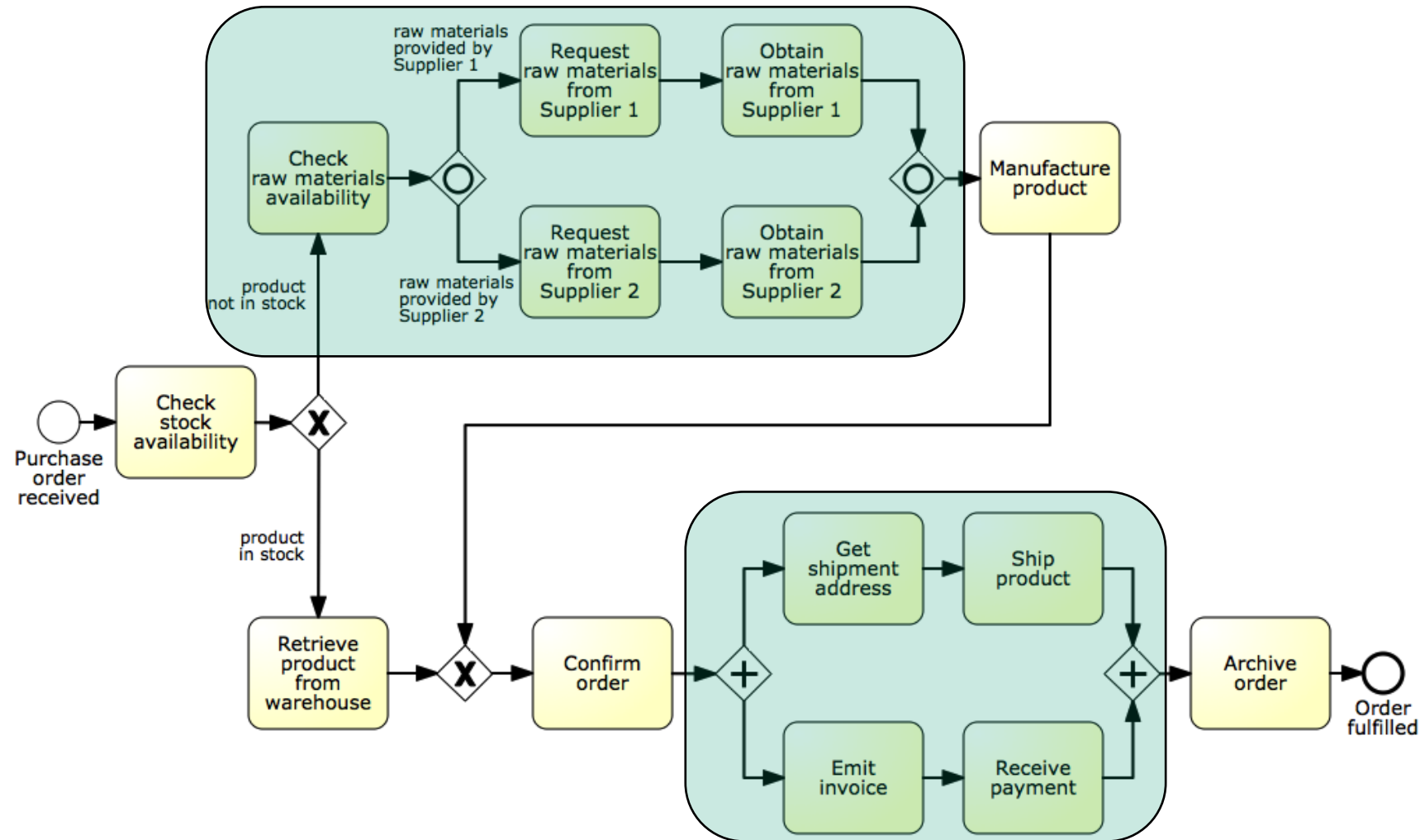


Does this look better?



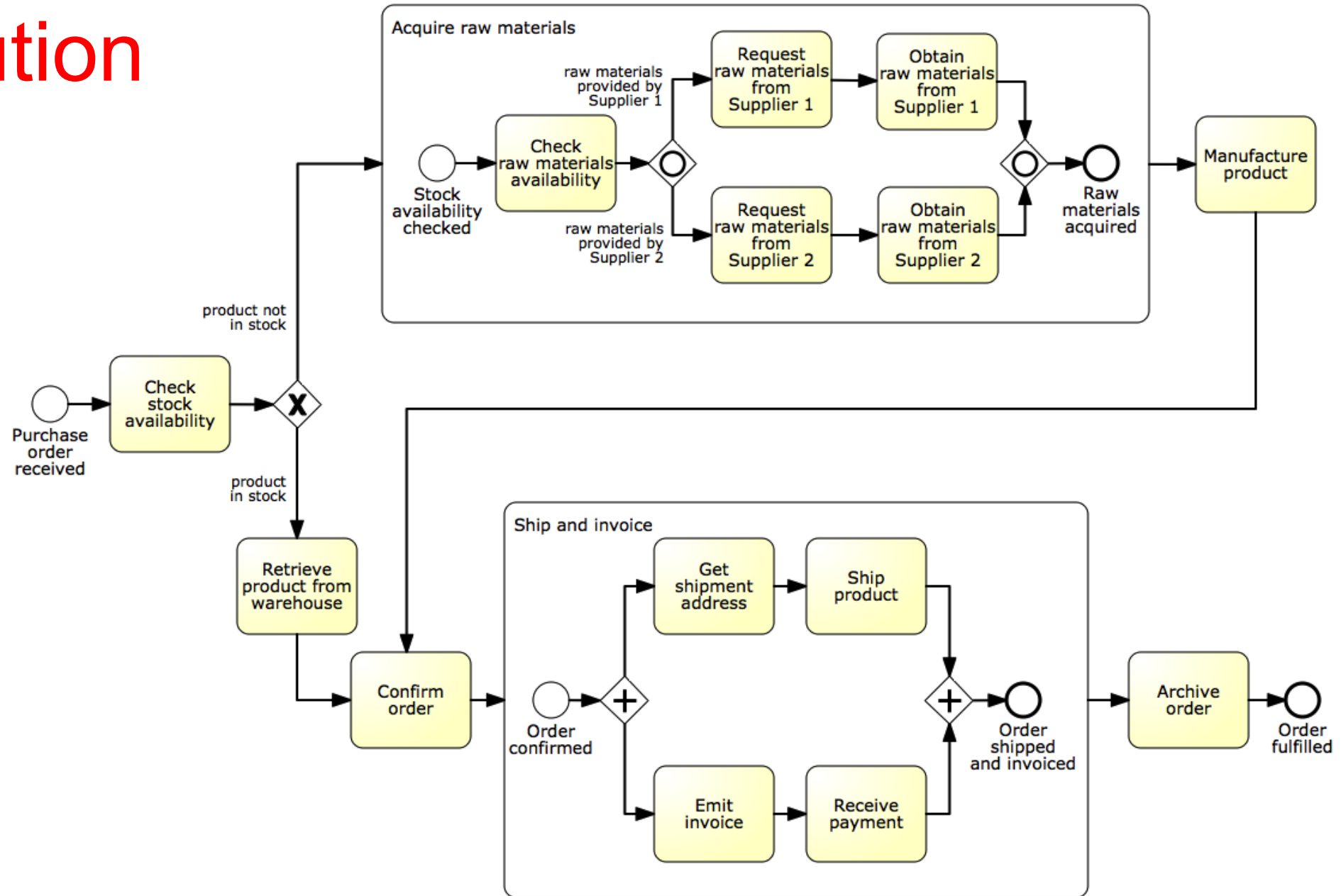
Identify possible sub-processes

Acquire raw materials

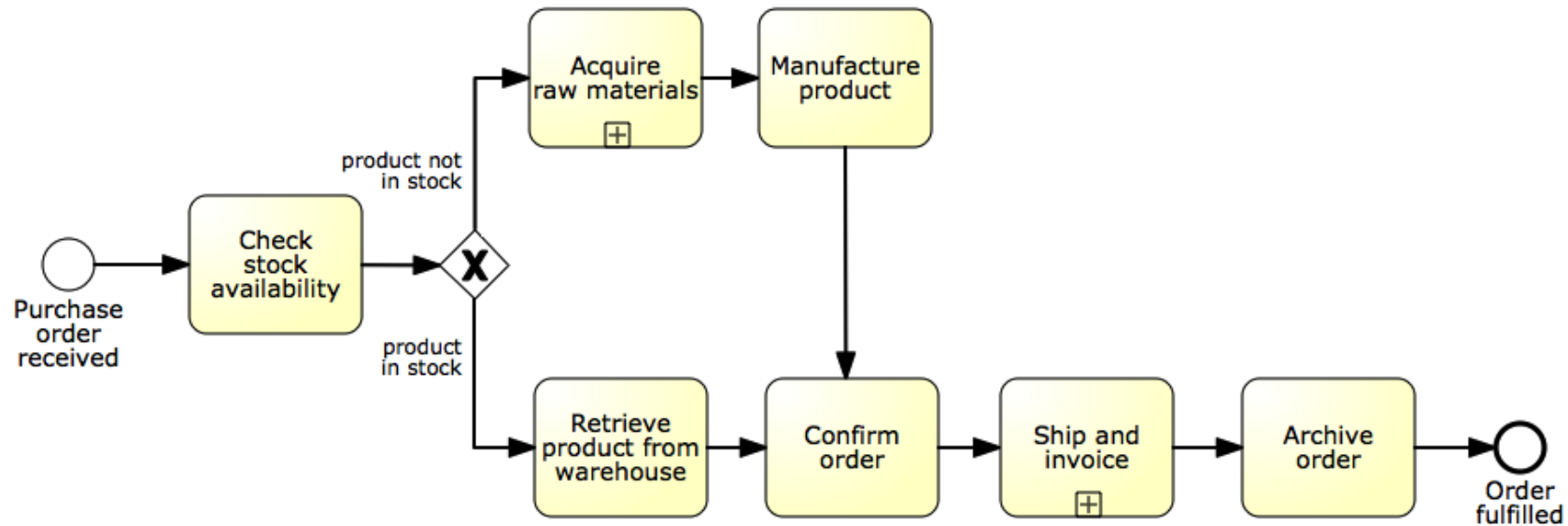


Ship and invoice

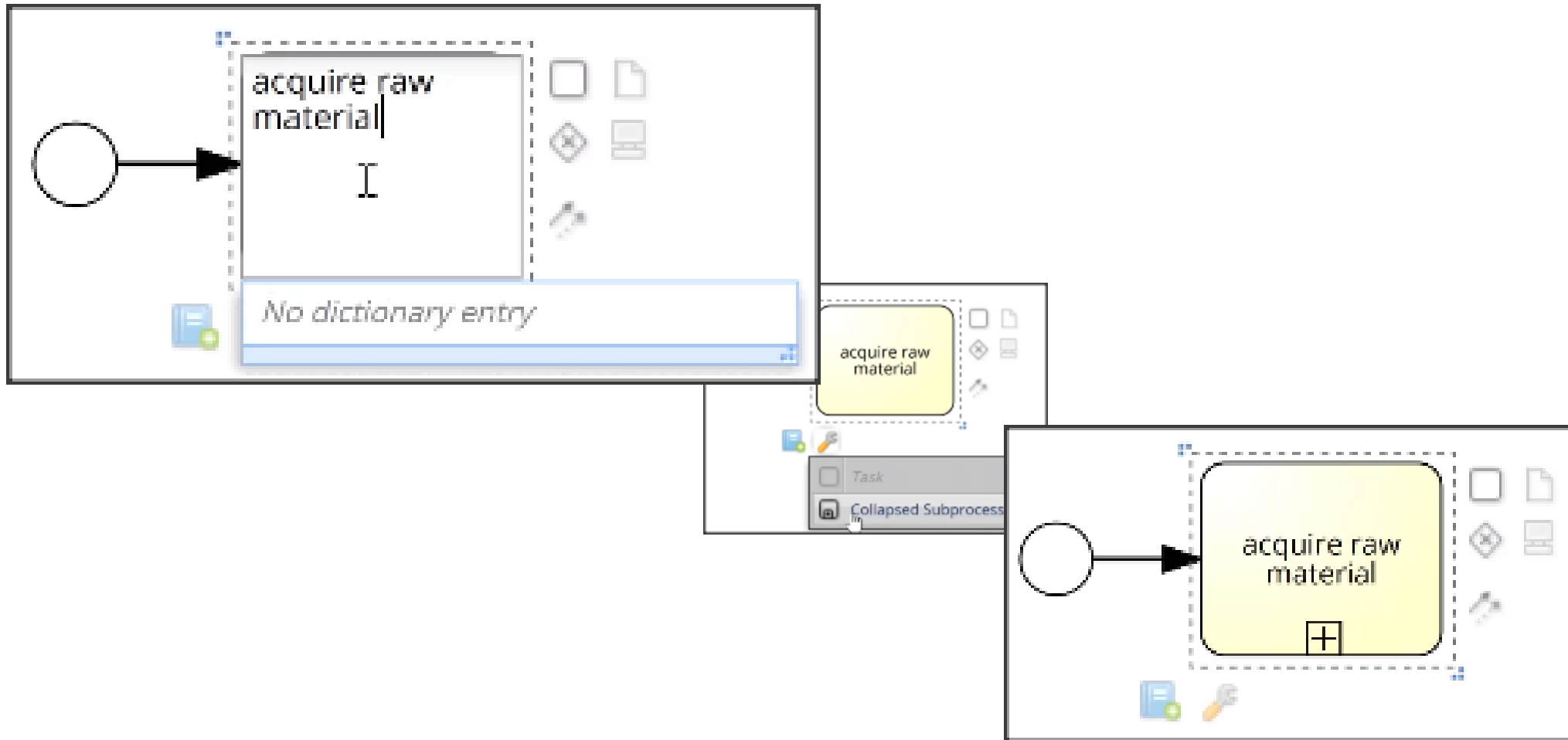
Solution



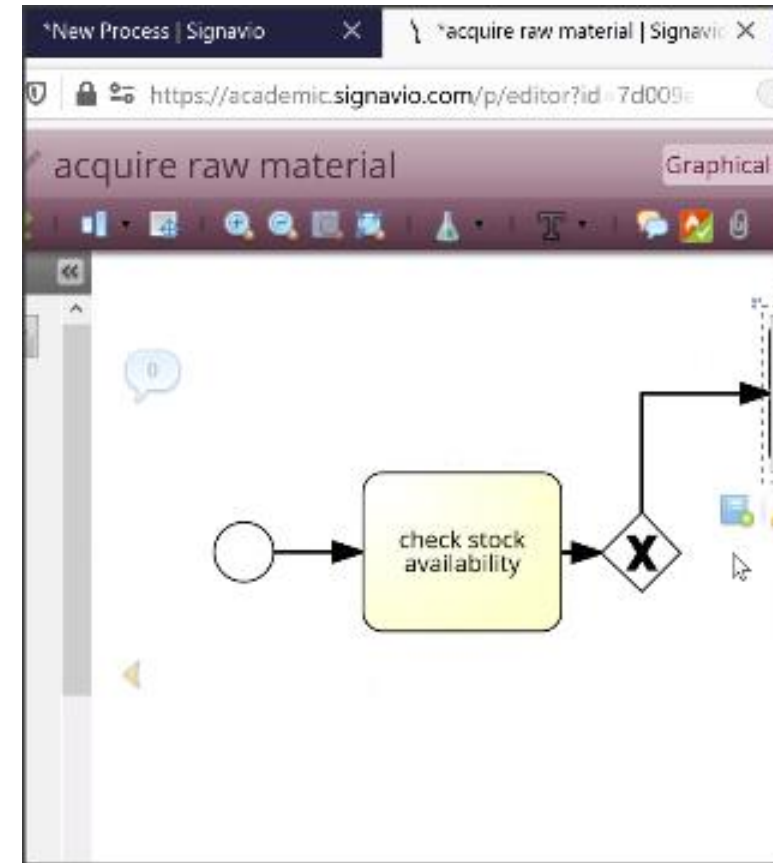
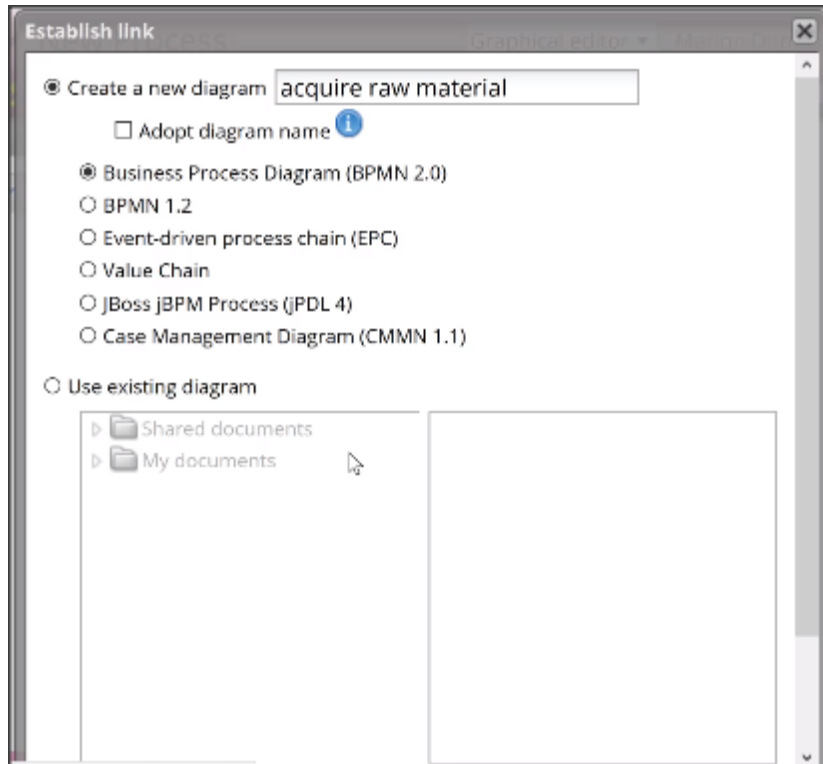
The refactored model



In signavio



Linking the sub-process in Signavio



Sub-processes

- ❑ An activity in a process can invoke a separate sub-process
- ❑ Use this feature to:
 1. Decompose large models into smaller ones, making them easier to understand and maintain

Guideline: Multi-level modeling

Level 1: value chain

- Simple linear description of the phases of the process
- No gateways
- Each activity chain is a sub-process

Level 2+: expand each activity in the value chain, add incrementally the following:

- Decisions, handoffs (lanes, pools)
- Parallel gateways, different types of events
- Data objects & data stores
- And as much detail as you need, and no more

Guideline: Multi-level modeling (cont.)

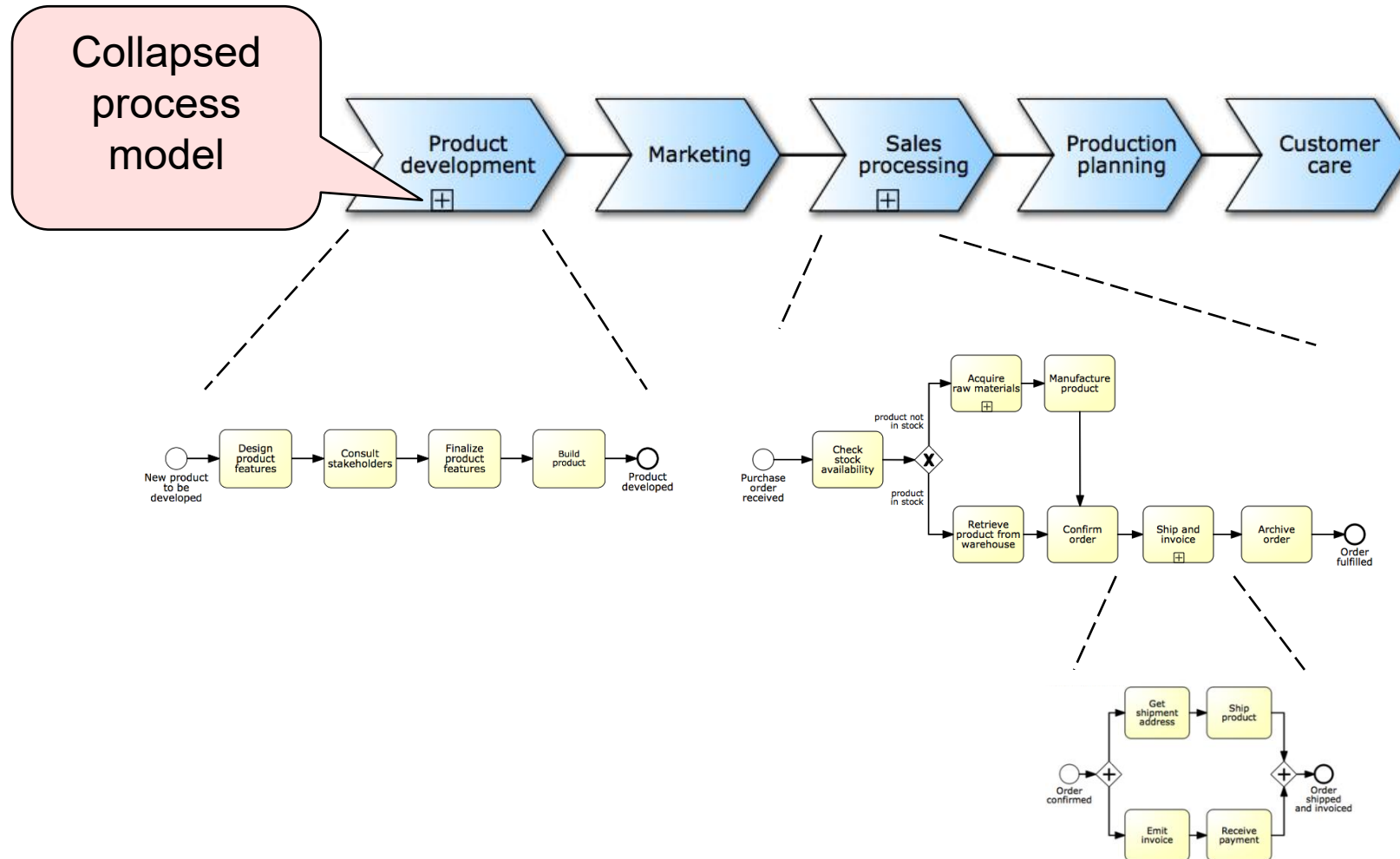
At each level, decompose according to:

- Logical milestones towards achieving the outcome of the process
- Major objects used in the process

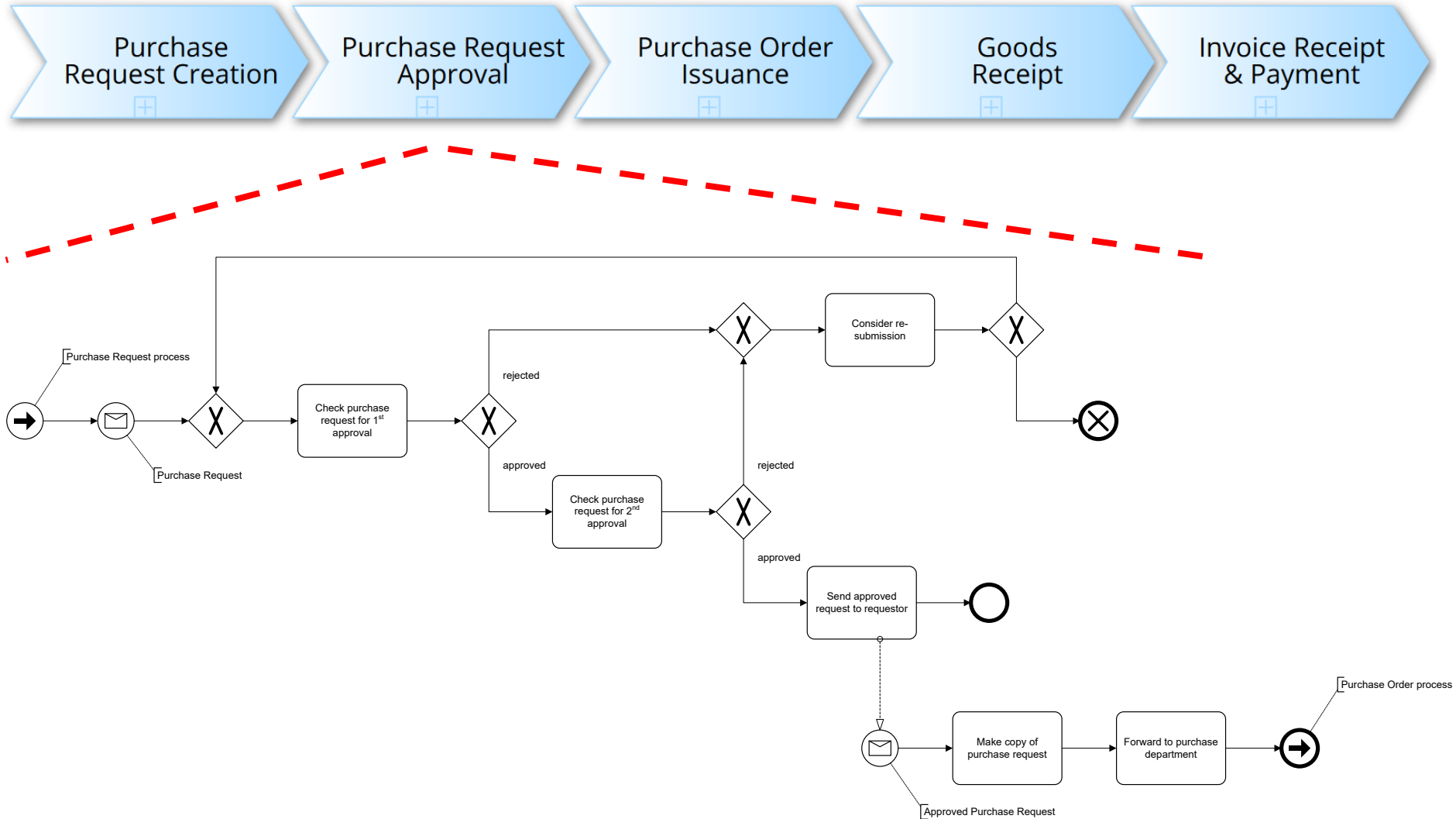
Decompose until processes are of “reasonable” size

- e.g. up to 20 nodes (tasks+events+gateways) per model

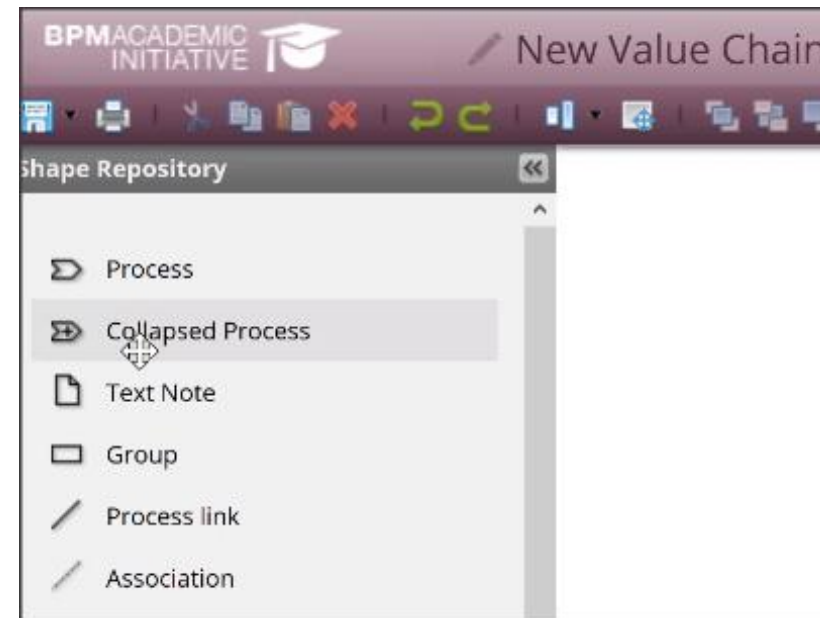
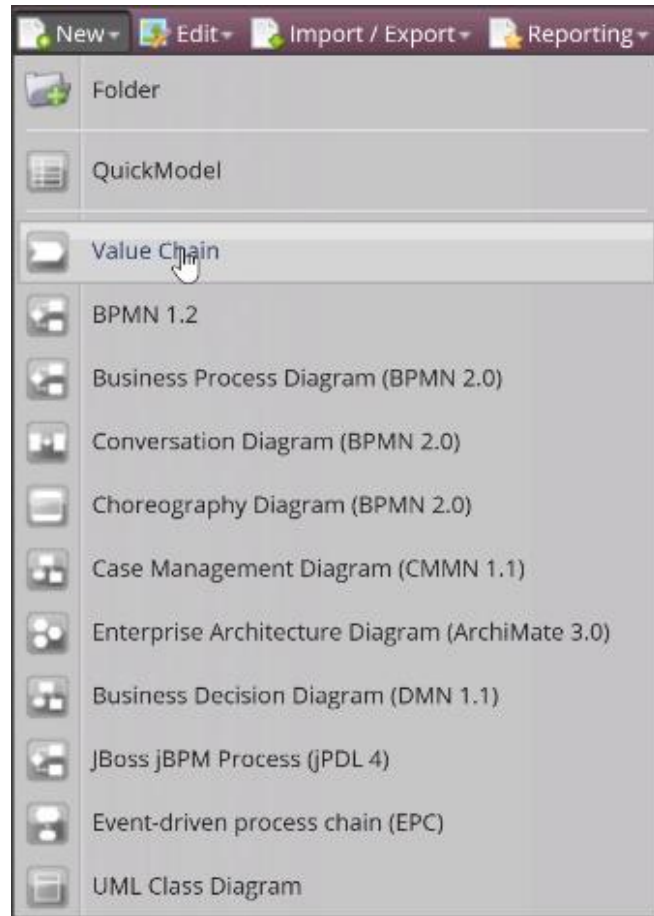
Non-BPMN value chain “chevron” notation - (e.g. Signavio, ARIS)



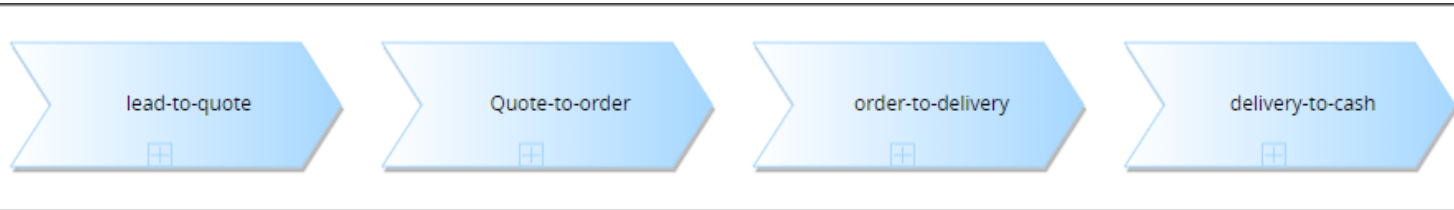
Value chain with sub-processes



Value Chain in Signavio



Value Chain in Signavio



Establish link

☒ Create a new diagram

☐ Adopt diagram name ⓘ

☐ Value Chain

☐ BPMN 1.2

☒ Business Process Diagram (BPMN 2.0)

☐ Choreography Diagram (BPMN 2.0)

☐ Conversation Diagram (BPMN 2.0)

☐ Event-driven process chain (EPC)

☐ Customer journey map

☐ Use existing diagram

▶ Shared documents

▶ My documents

When should we decompose a process model into sub-processes?



Process Decomposition

When should we decompose a process model into sub-processes?

This then begs the question:

- What are the quality attributes of a process model?
 - Readability
 - Understandability
 - Accuracy
 - Completeness

Use sub-processes when the model becomes too large:

- Hard to understand
- Increased error probability

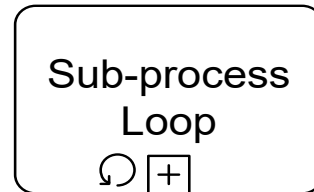
Week 3: Advanced Process Modeling

Contents

- BPMN main elements (Recap of week 2)
- Process Decomposition (Chp 3)
- **Rework and Repetition (Chp 4)**
- Handling Events (Chp 4)
- Handling Exceptions (Chp 4)
- Recap

Block-structured repetition: Loop Activity

BPMN also provides the *loop activity* construct to allow the repetition of a task or sub-process



George Joukhadar [Try the new Editor](#)

Attributes (Task)

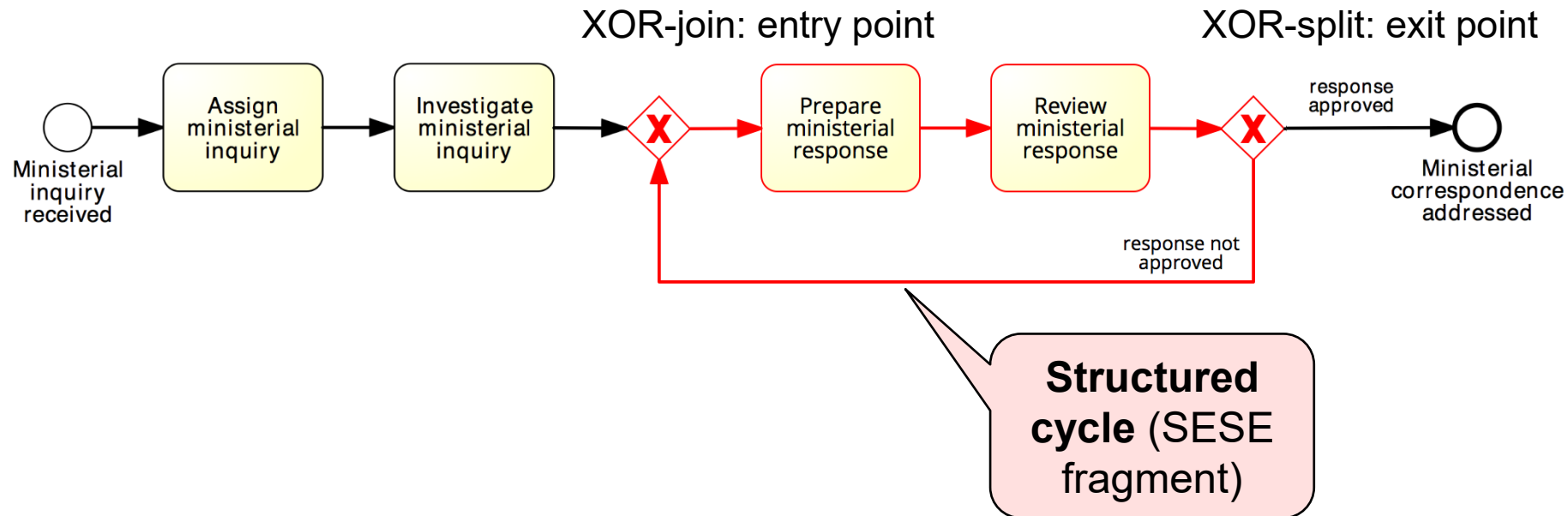
Attribute	Value
Main Attributes	
Name	
Documentation	
Is a call activity	
Task type	None
Background color	
Loop type	None
Border color	None
Cost & Resource A	
More Attributes	
Is for compensation	

Standard

MI Parallel

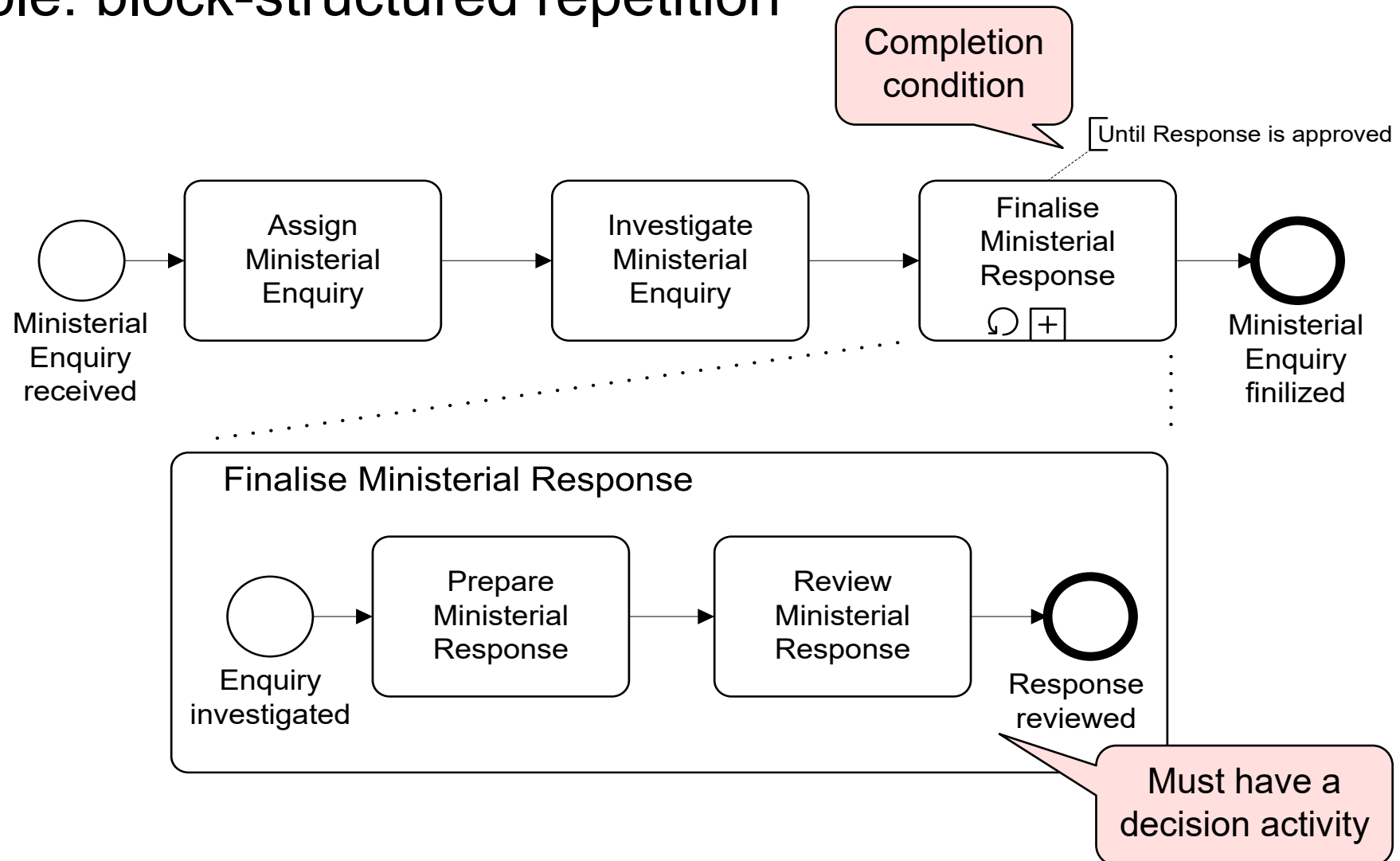
MI Sequential

More on rework and repetition



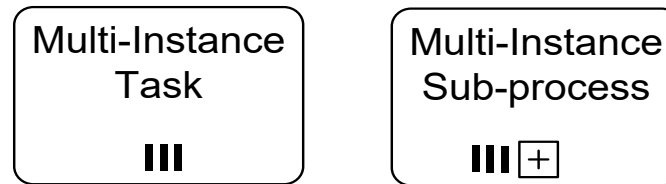
SESE = Single Entry Single Exit fragment, i.e. a fragment delimited by a single entry node and a single exit node (there are no other incoming arcs into the fragment or outgoing arcs from the fragment)

Example: block-structured repetition



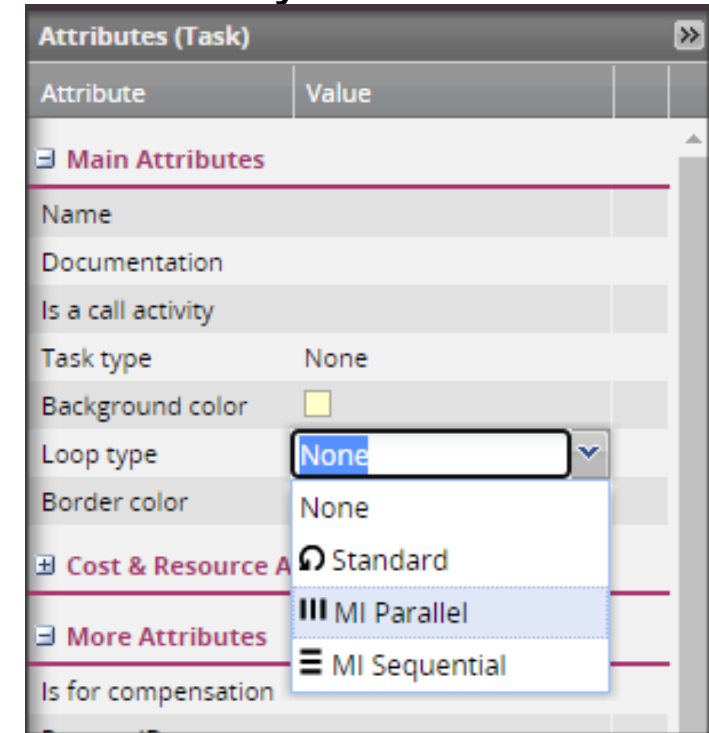
Parallel repetition: multi-instance activity

The multi-instance activity provides a mechanism to indicate that an activity is executed *multiple times concurrently*



Useful when the same activity needs to be executed for multiple entities or data items, such as:

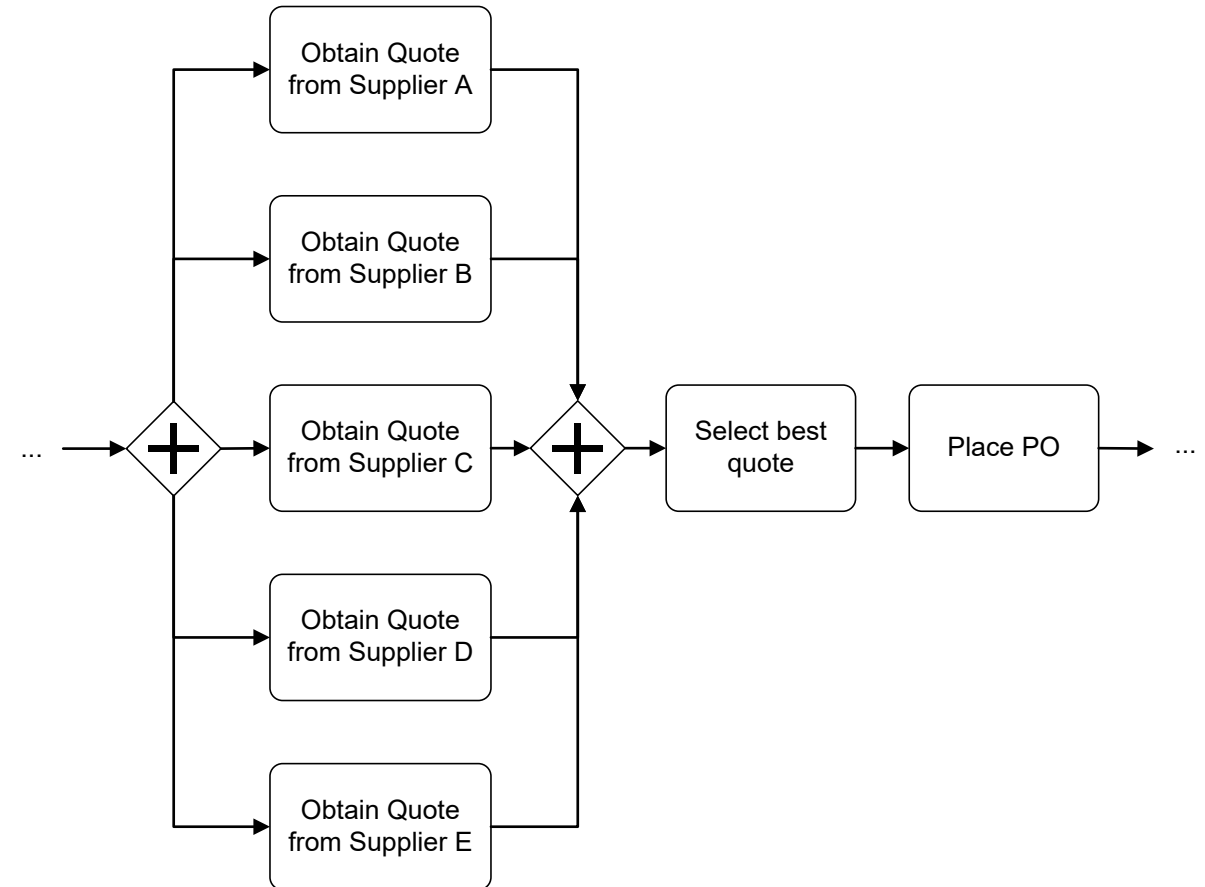
- Request quotes from multiple suppliers
- Check the availability for each line item in an order separately
- Send and gather questionnaires from multiple witnesses in the context of an insurance claim



Example: multi-instance activity

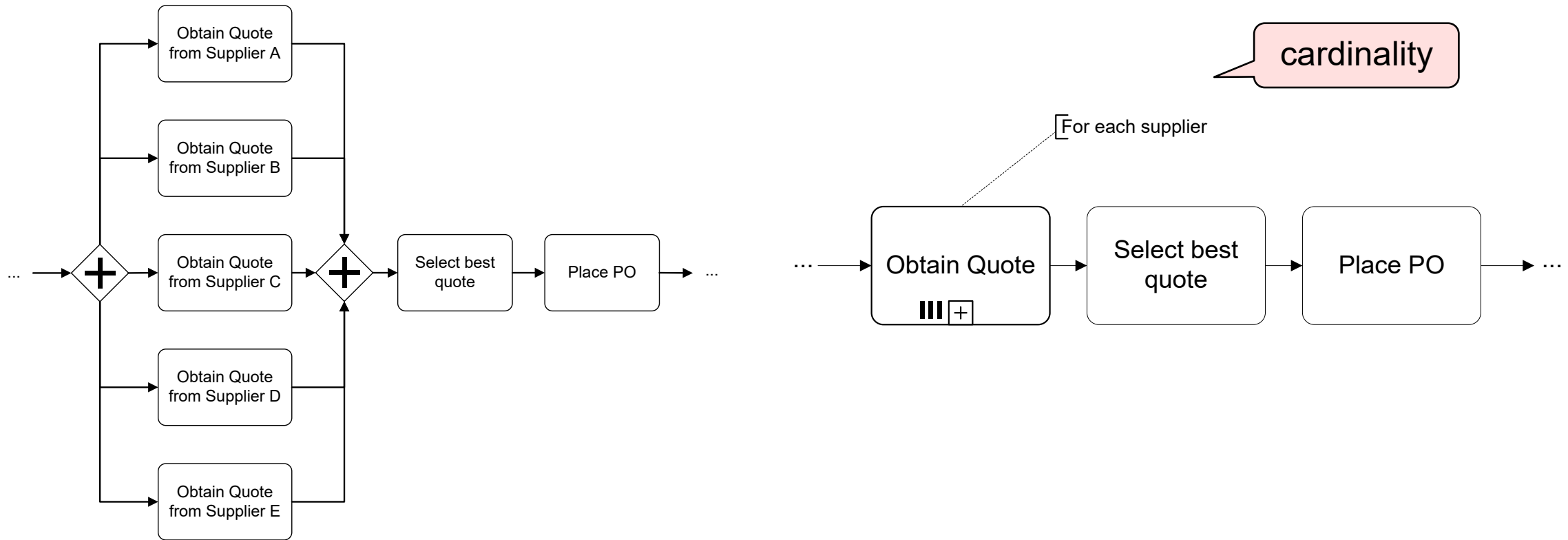
Procurement

In procurement, typically a quote is to be obtained from all preferred suppliers (assumption: five preferred suppliers exist). After all quotes are received, they are evaluated and the best quote is selected. A corresponding purchase order is then placed.



With multi-instance activity - Solution

Procurement



Week 3: Advanced Process Modeling

Contents

- BPMN main elements (Recap of week 2)
- Process Decomposition (Chp 3)
- Rework and Repetition (Chp 4)
- **Handling Events (Chp 4)**
- Handling Exceptions (Chp 4)
- Recap

Events handling

In BPMN, events model something instantaneous happening during the execution of a process

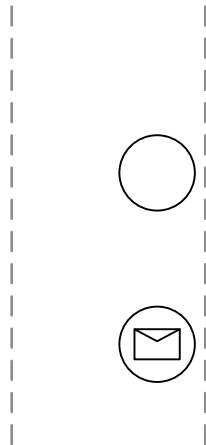
They affect the process flow:

- Start
- **Intermediate**
- End



BPMN event types

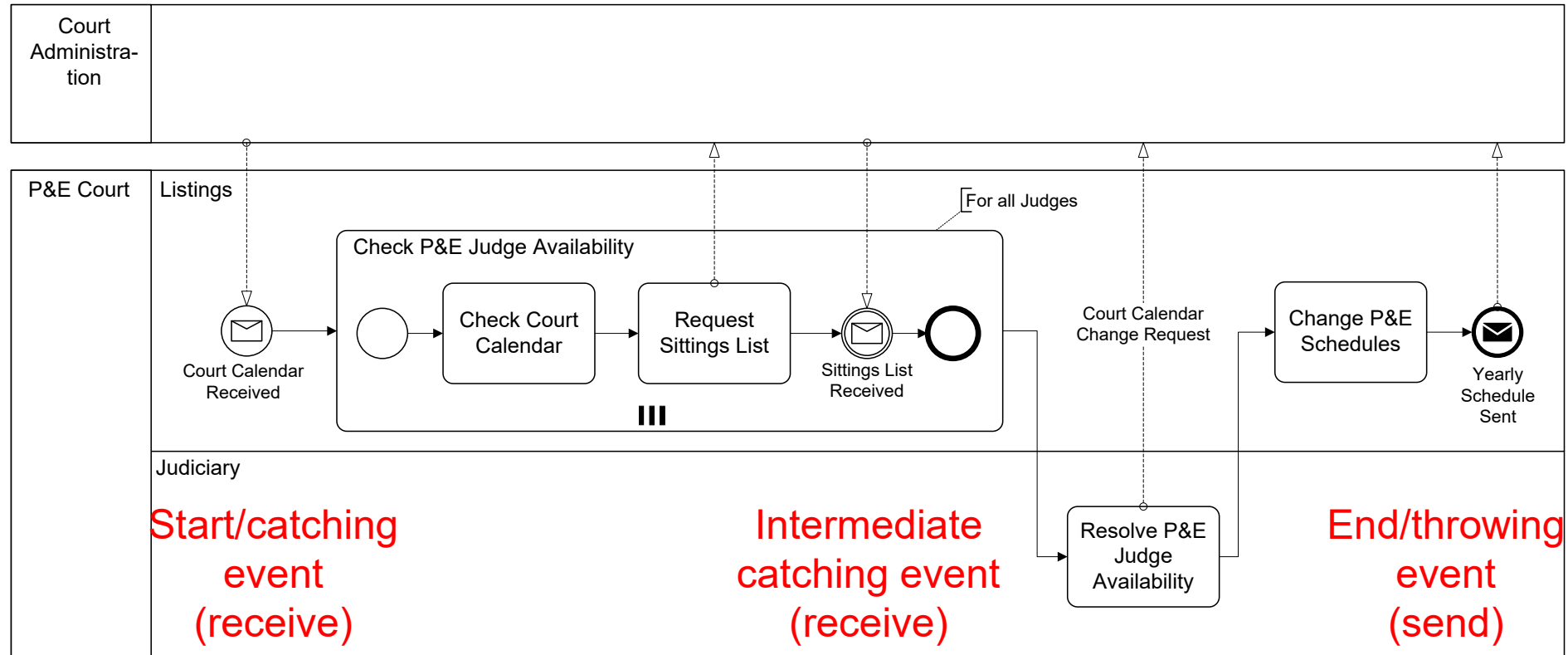
Start Intermediate End



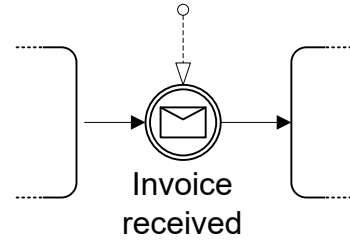
Untyped Event – Indicates that an instance of the process is created (start) or completed (end), without specifying the cause for creation/completion

Start Message Event – Indicates that an instance of the process is created when a message is **received**

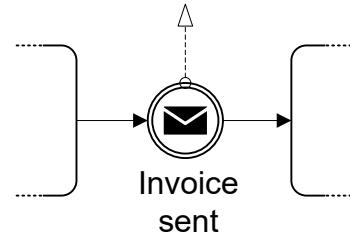
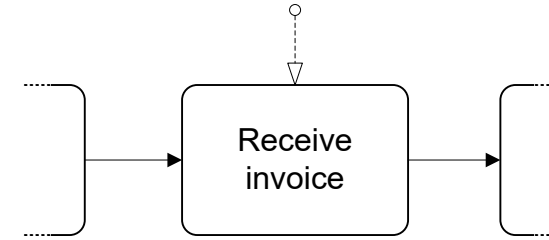
Example: message events



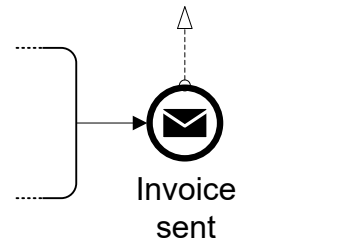
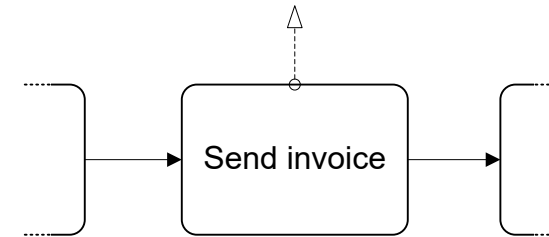
Comparison with sending/receiving tasks



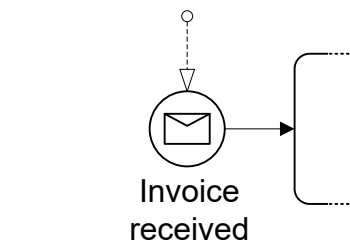
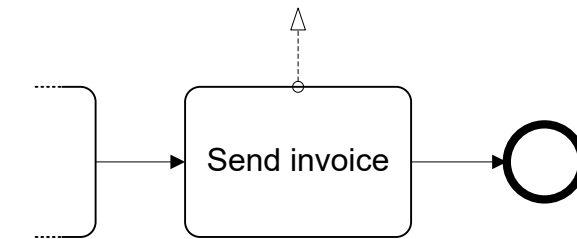
=



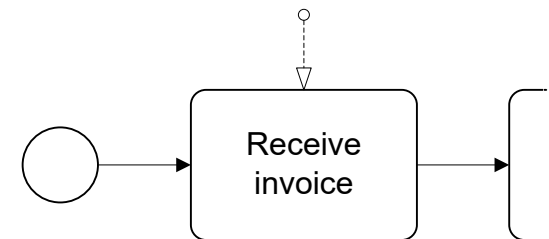
=



=

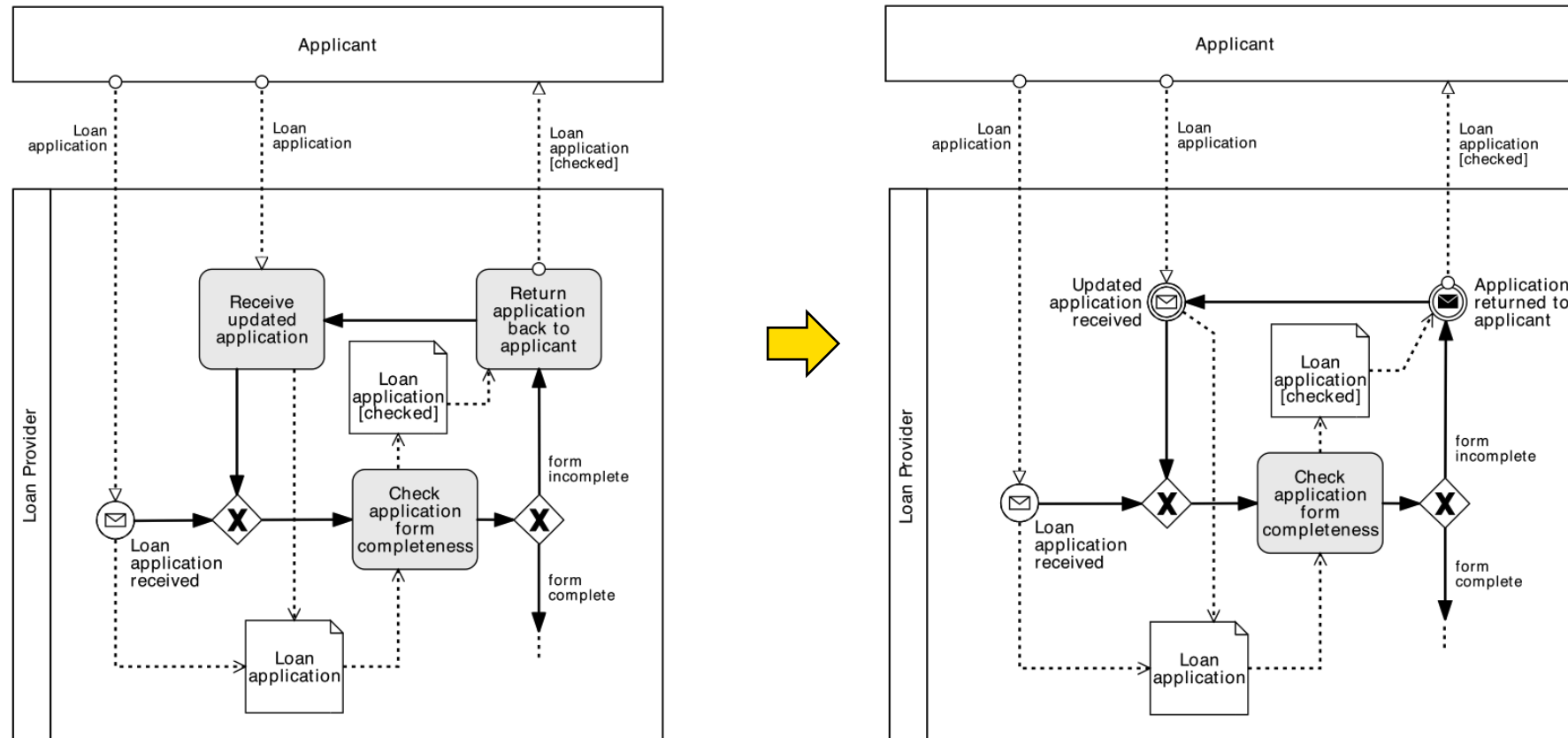


≠



So, when to use what?

Use message events only when the corresponding activity would simply send or receive a message and do nothing else (*i.e. when activity does not really represent a unit of work*)



Temporal events (timer events)

Start Intermediate End



Start Timer Event – Indicates that an instance of the process is created at certain date(s)/time(s), e.g. start process at 6pm every Friday



Intermediate Timer Event – Triggered at certain date(s)/time(s), or after a time interval has elapsed since the moment the event is “enabled” (delay)

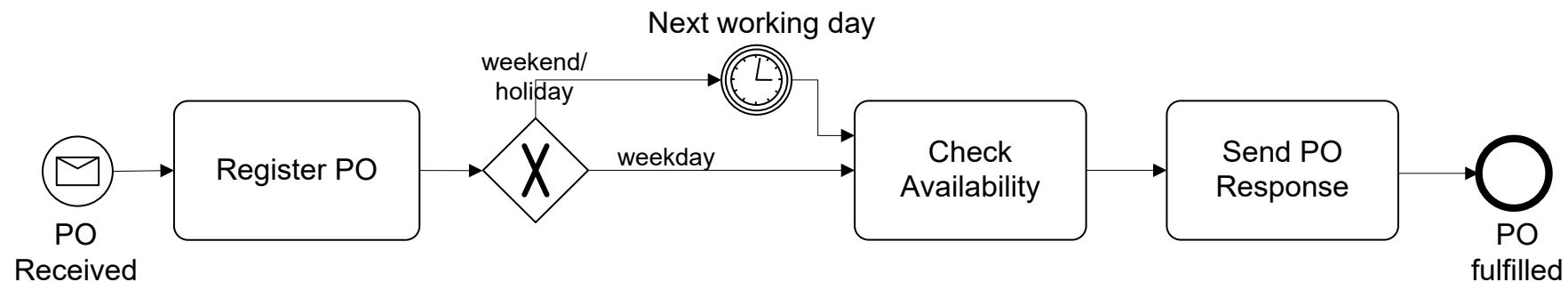
Timer events are catching events only

The process does not generate the timer, but reacts to it

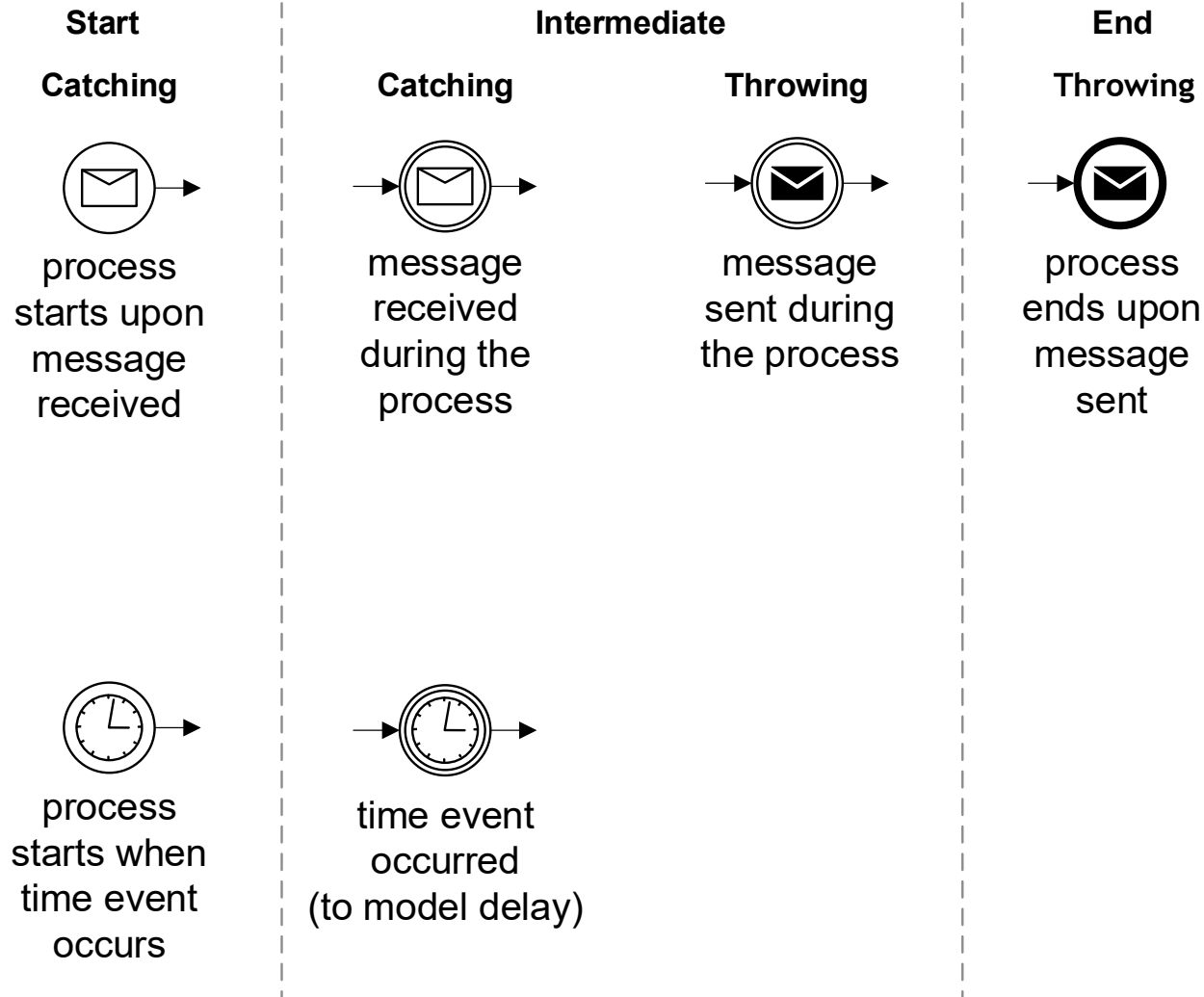
Coming back to our scenario...

PO handling

A Purchase Order (PO) handling process starts when a **PO is received**. The PO is first registered. If the current date is not **a working day**, the process waits until the **following working day** before proceeding. Otherwise, an availability check is performed and a **PO response is sent** back to the customer.



Recap: Message and Timer events



Racing events: Event-based decision

With the XOR-split gateway, a branch is chosen based on **conditions** that evaluate over available **data**

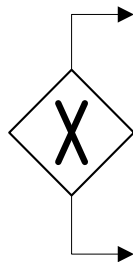
→ The choice can be made immediately after the token arrives from the incoming flow

Sometimes, the choice must be delayed until an **event happens**

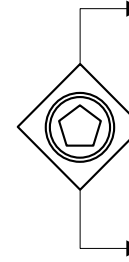
→ The choice is based on a “**race**” among events

Two types of XOR split:

data-driven
XOR-split



event-driven
XOR split



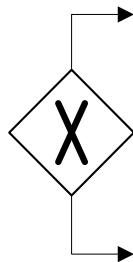
Empty pentagon
enclosed in a
double-line circle

Exercise 4.6: Stock replenishment

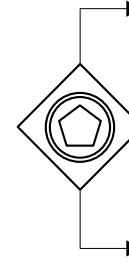
Model the following process.

*A restaurant chain submits a purchase order (PO) to replenish its warehouses every Thursday. The restaurant chain's procurement system **expects to receive either a "PO Response" or an error message**. However, it may also happen that **no response is received at all** due to system errors or due to delays in handling the PO on the supplier's side. If no response is received by Friday afternoon or if an error message is received, a purchasing officer at the restaurant chain's headquarters should be notified. Otherwise, the PO Response is processed normally.*

data-driven
XOR-split

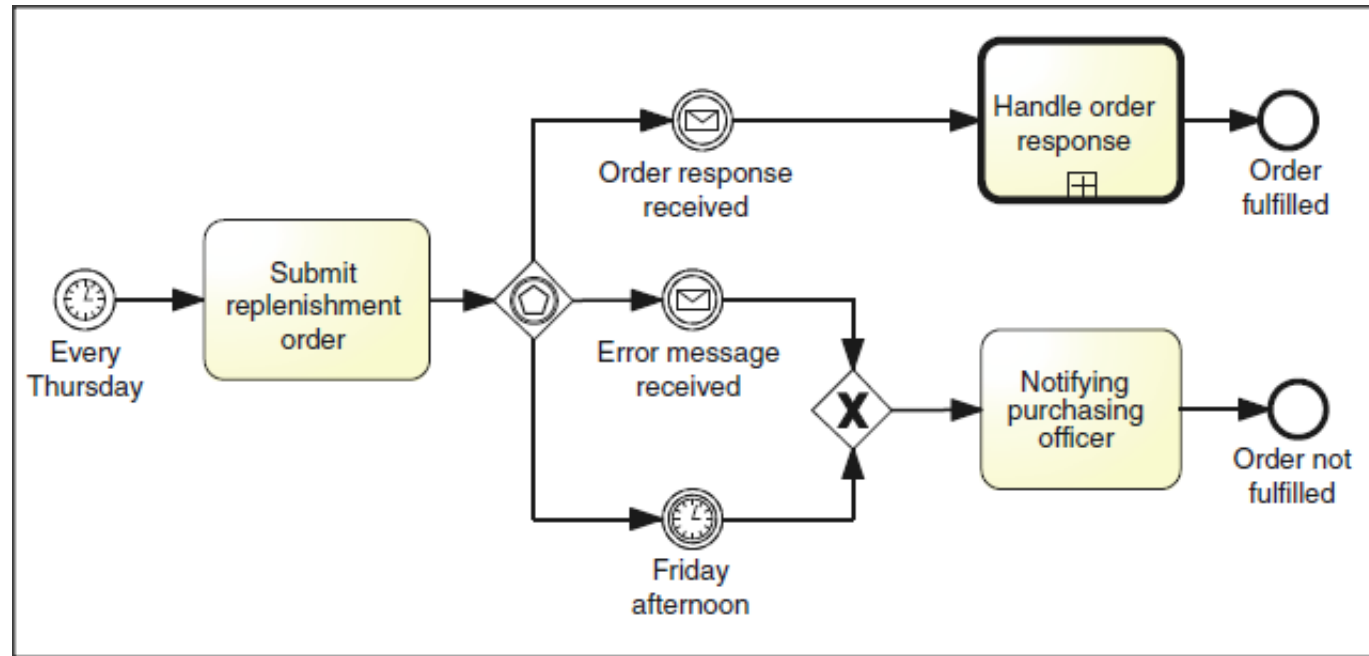


event-driven
XOR split

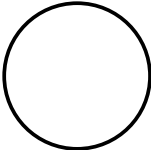
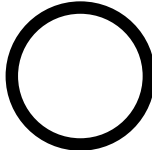
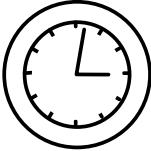


Empty pentagon
enclosed in a
double-line circle

Exercise 4.6: Stock replenishment - Solution



Recap: message and timer events

	Start	Intermediate		End
		Catching	Throwing	
Untyped: indicate start point, state changes or final states.				
Message: Receiving and sending messages.				
Timer: Cyclic timer events, points in time, time spans or timeouts.				

Week 3: Advanced Process Modeling

Contents

- BPMN main elements (Recap of week 2)
- Process Decomposition (Chp 3)
- Rework and Repetition (Chp 4)
- Handling Events (Chp 4)
- Handling Exceptions (Chp 4)
- Recap

Process abortion

Exceptions are events that deviate a process from its “normal” course

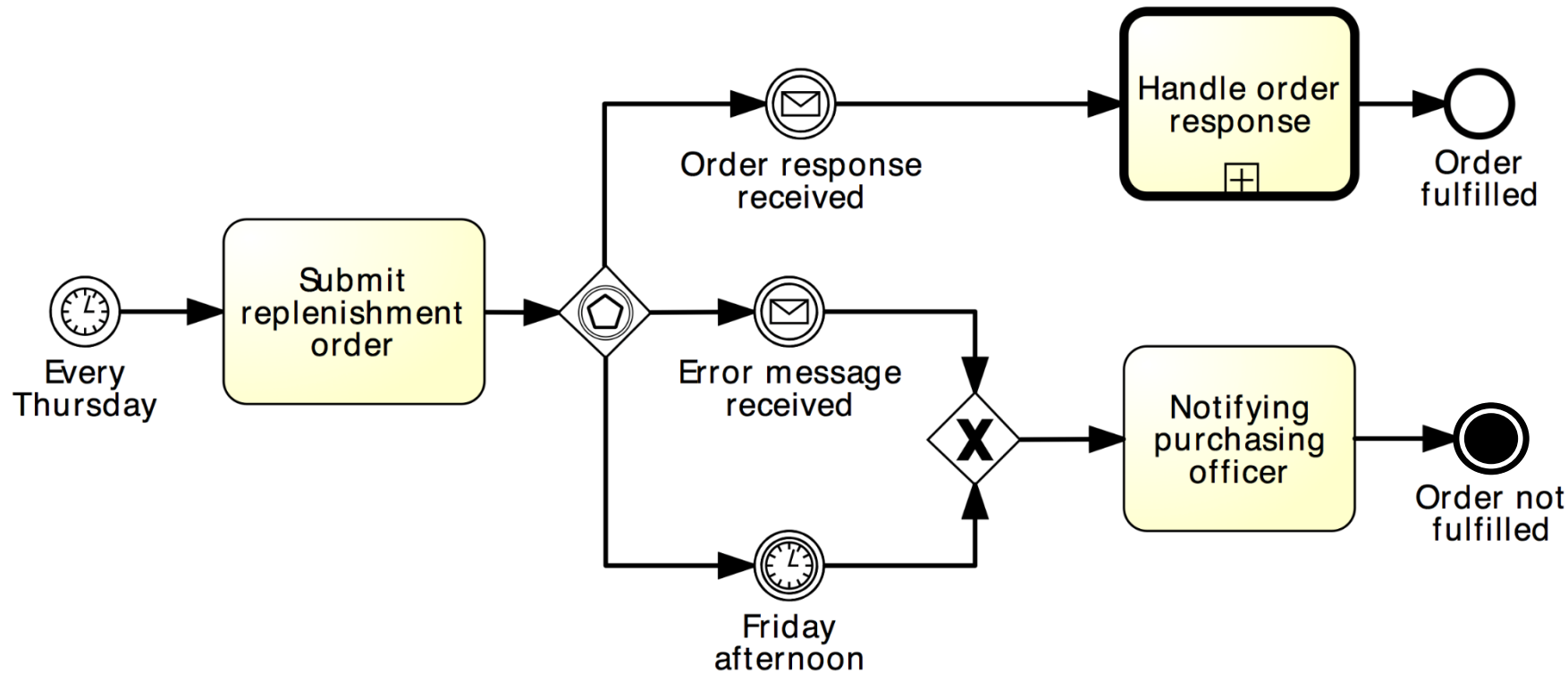
The simplest form of exception is to notify that there is an exception (negative outcome)

This can be done via the Terminate end event: **it forces the whole process to abort** (“wipes off” all tokens left behind, if any)



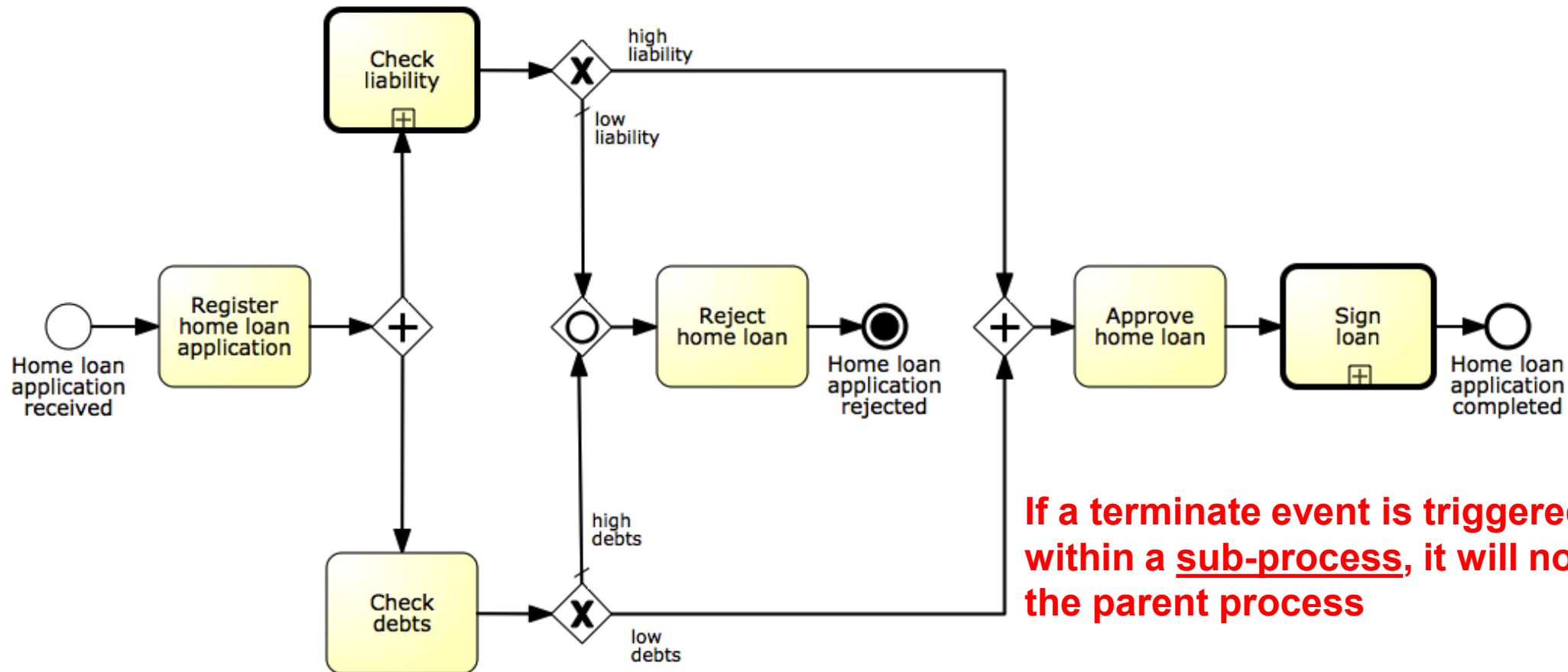
Example 1: terminate event

Signal the negative outcome...



Example 2: terminate event

Abort the process by removing all tokens...

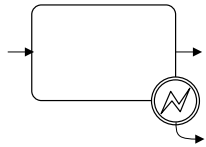


If a terminate event is triggered from within a sub-process, it will not abort the parent process

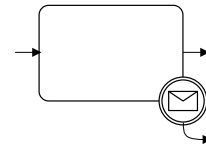
Exception handling

Handling exceptions often involves stopping a sub-process and performing a special activity

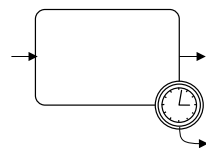
Types of exceptions for an activity (task/sub-process):



Internal: something goes wrong inside an activity, whose execution must thus be interrupted. Handled with the **Error event**



External: something goes wrong outside the process, and the execution of the current activity must be interrupted. Handled with the Message event (e.g **customer cancelled the order**)



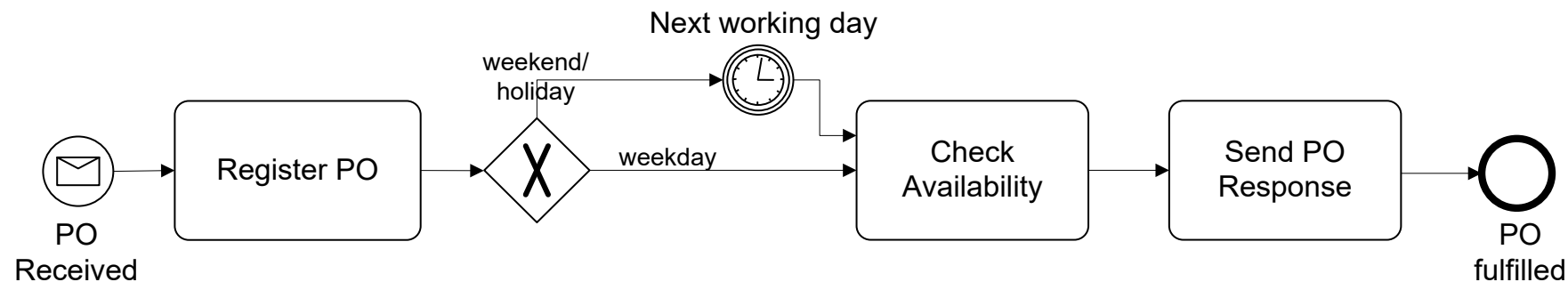
Timeout: an activity takes too long and must be interrupted. Handled with the Timer event

All these events are catching intermediate events. They stop the enclosing activity and start an exception handling routine.

Let's extend our PO handling process

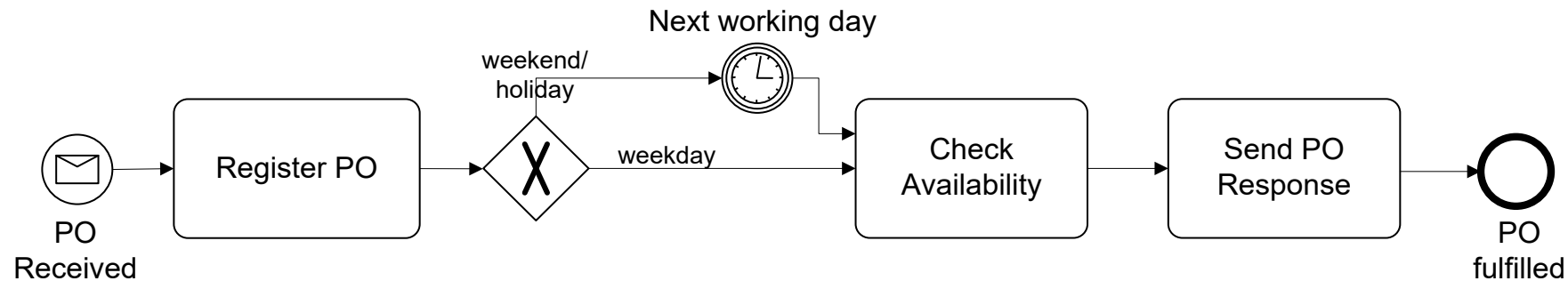
PO handling

A PO handling process starts when a PO is received. The PO is first registered. If the current date is not a working day, the process waits until the following working day before proceeding. Otherwise, an availability check is performed and a PO response is sent back to the customer.



Let's extend our PO handling process

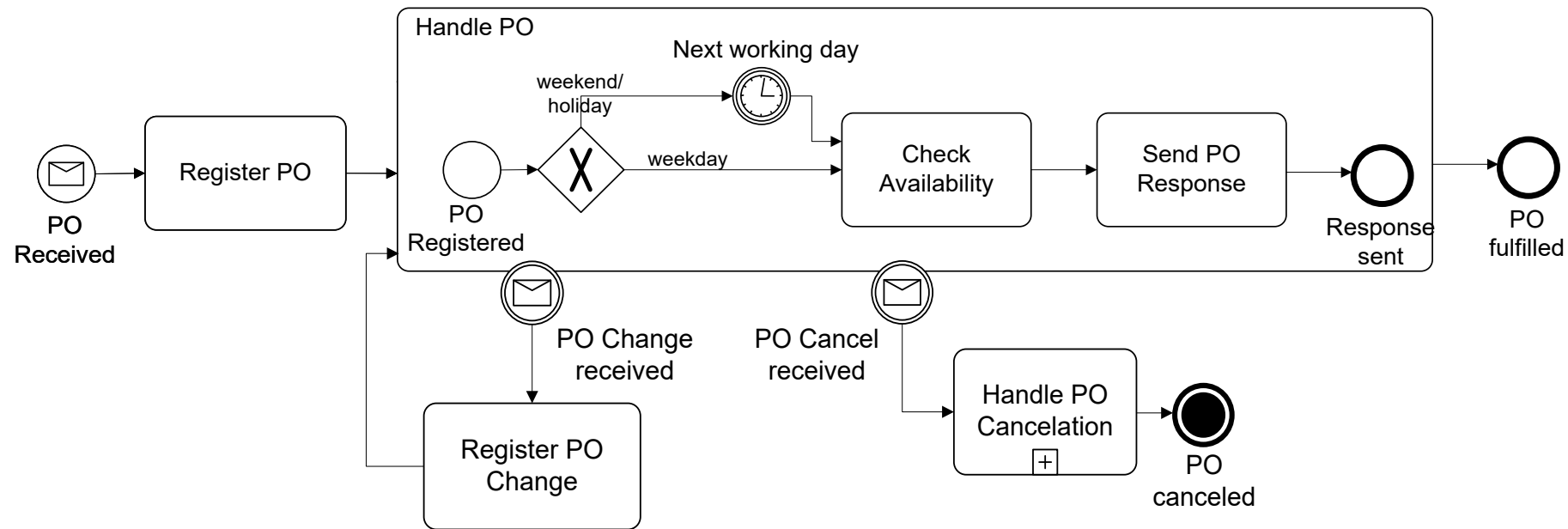
PO handling



A PO **change request** may be received anytime after the PO is registered. This request includes a change in quantity or line items. When such a request is received, any processing related to the PO must be stopped. The PO change request is then registered. Thereafter, the process proceeds as it would do after a normal PO is registered. Further, if the customer sends a PO **cancellation request after the PO registration**, the PO processing must be stopped and the cancellation request must be handled.

Solution: exception handling

PO handling



Internal exception: error event

Start Intermediate End



Error Event – Indicates an error: the “end” version generates an error event while the “catching intermediate” version consumes it when attached to the boundary of an activity

Must be attached to the activity's boundary



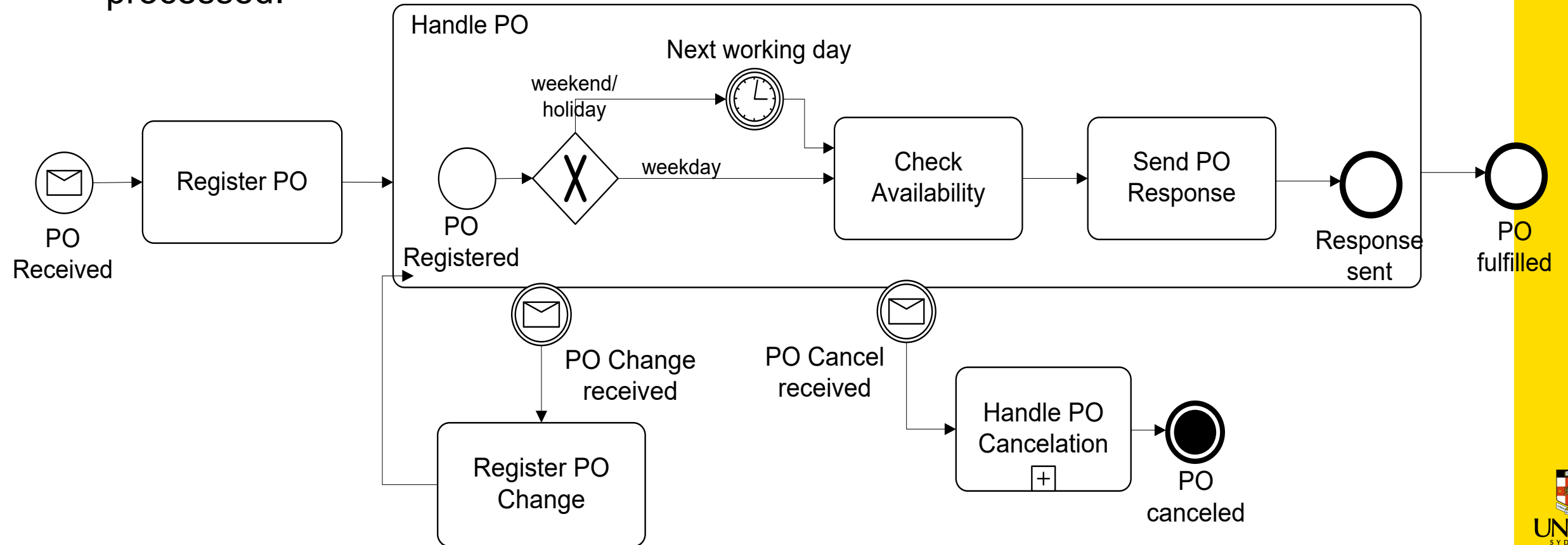
Example: internal exception

PO handling

Consider again our “PO Handling process” example with the following extension:

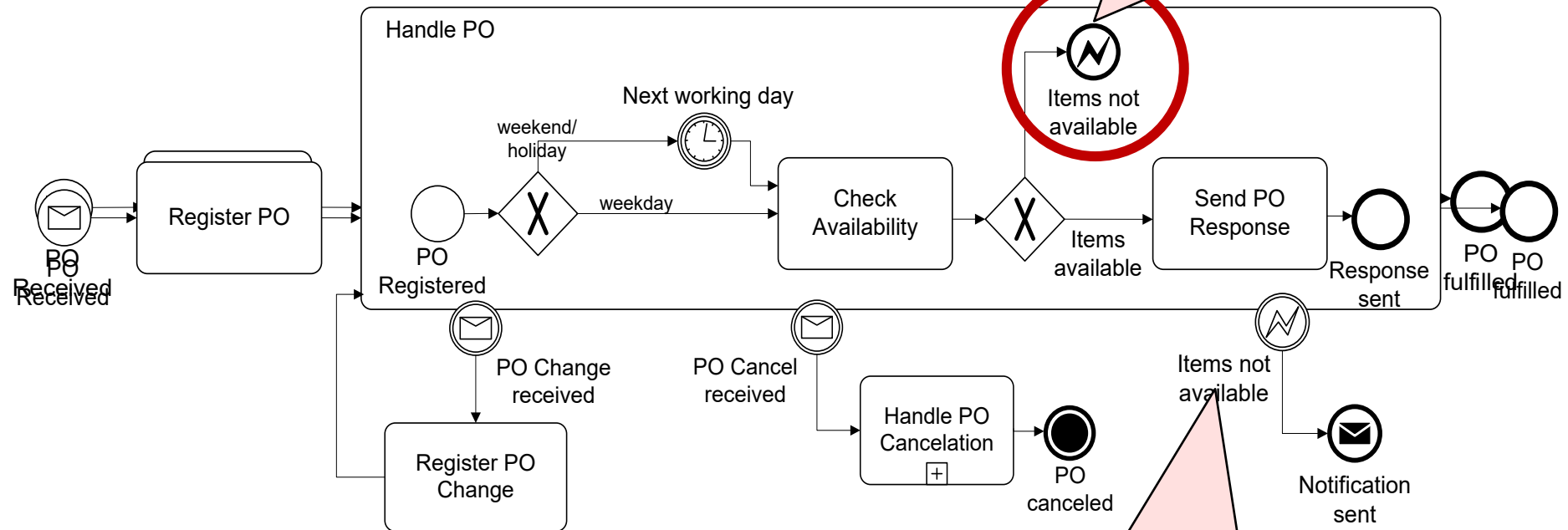
if an item is not available, any processing related to the PO must be stopped.

Thereafter, the client needs to be notified that the PO cannot be further processed.



Solution: internal exception

PO handling



Throwing and catching error events must have the **same** label

Must catch an error event thrown from **within** the same activity

Exercise 4.10: activity timeout

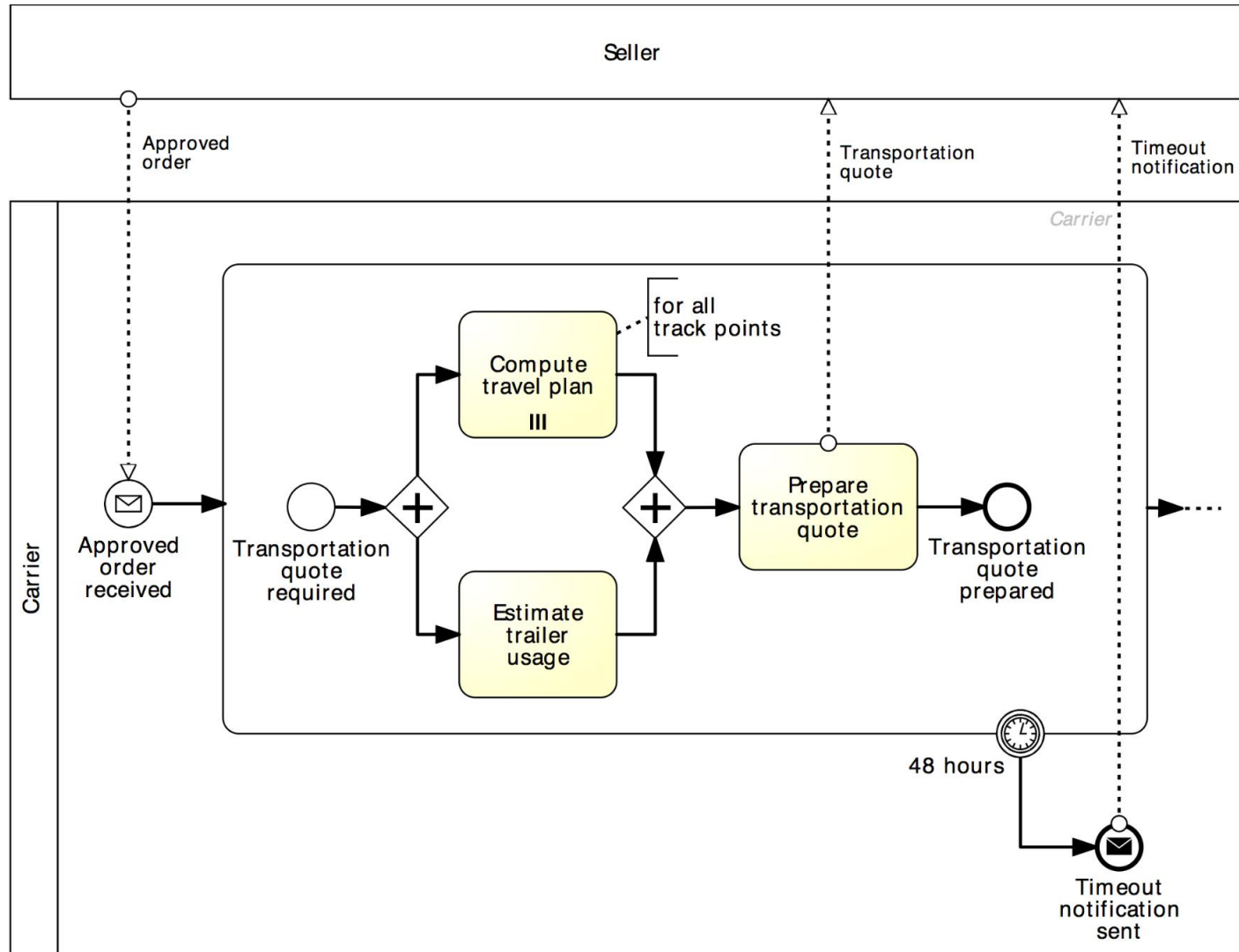
Order-to-transportation quote

Once a wholesale order has been confirmed, the supplier transmits this order to the carrier for the preparation of the transportation quote. In order to prepare the quote, the carrier needs to compute the route plan (including all track points that need to be traversed during the travel) and estimate the trailer usage.

By contract, wholesale orders have to be dispatched within four days from the receipt of the order. This implies that transportation quotes have to be prepared within 48 hours from the receipt of the order to remain within the terms of the contract.

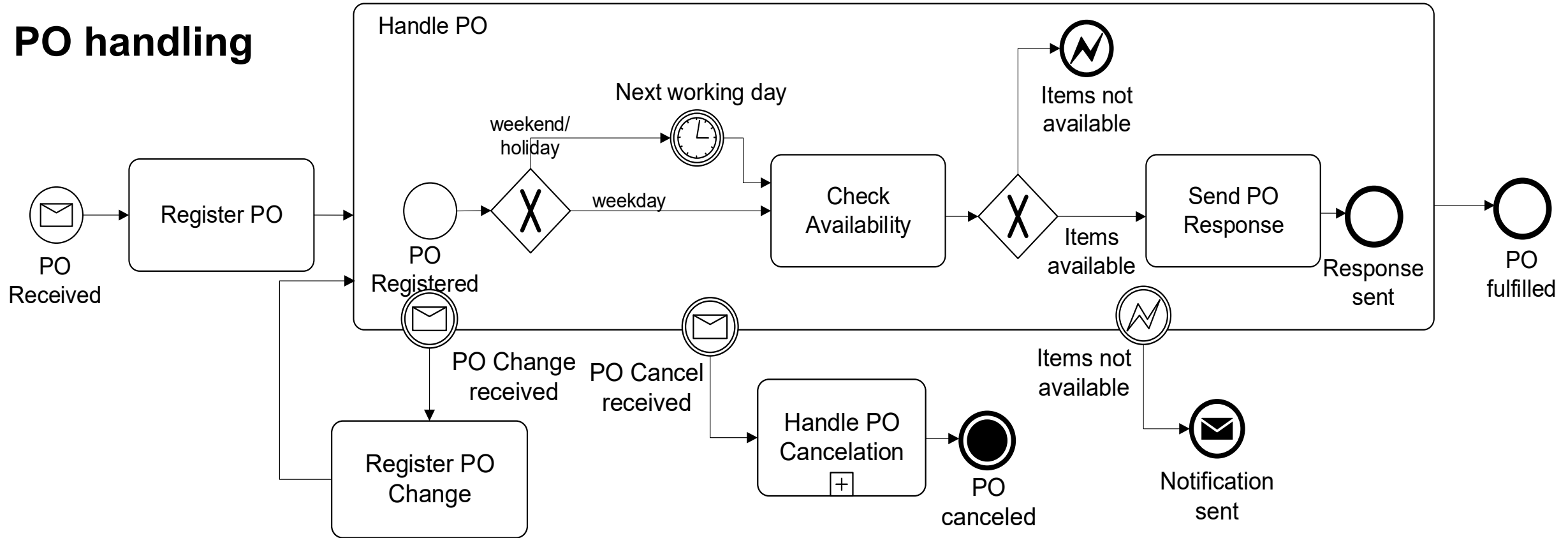
Solution: activity timeout

Order-to-transportation quote



More on the PO handling example...

PO handling

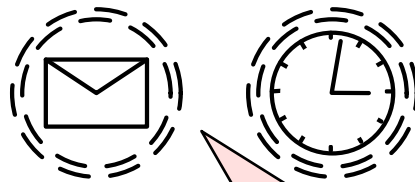


The customer may send a request for address change after the PO registration. When such a request is received, **it is just registered**, without further action.

Non-interrupting boundary events

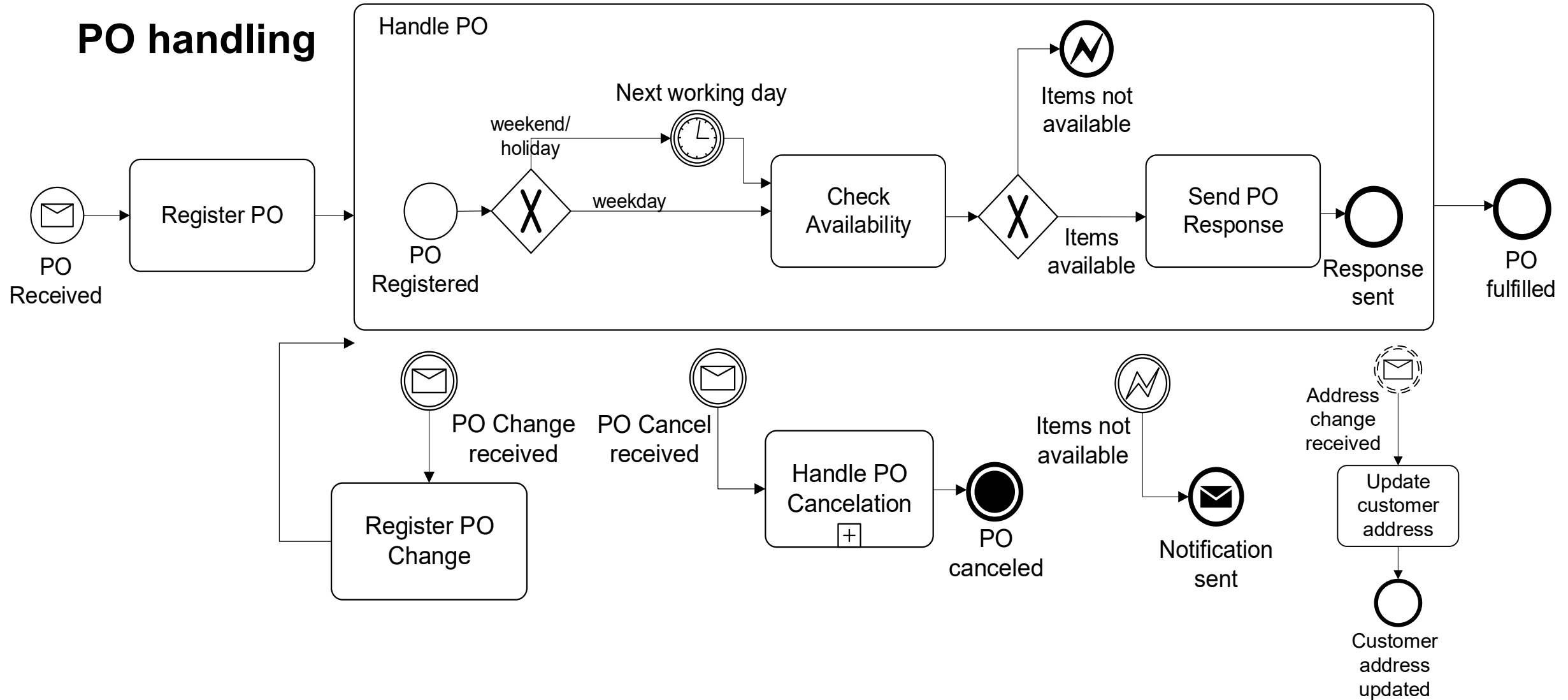
Sometimes we may need to trigger an activity **in parallel** to the normal flow, i.e. without interrupting the normal flow.

This can be achieved by using *non-interrupting* boundary events



Must be attached to
the activity's
boundary

Solution: non-interrupting boundary events



Week 3: Advanced Process Modeling

Contents

- BPMN main elements (Recap of week 2)
- Process Decomposition (Chp 3)
- Rework and Repetition (Chp 4)
- Handling Events (Chp 4)
- Handling Exceptions (Chp 4)
- **Recap**

Recap

- In this lecture we have learned about:
 - BPMN sub-processes
 - Repetition markers: loop marker and parallel multi-instance marker
 - Events: timer, message and error events
 - Event-based choice gateway
 - Boundary events: interrupting and non-interrupting
 - Error events (throw and catch)